

Controlling Reasoning Effort in LLMs

How LLMs Learn Low-, Medium-, and High-Effort Reasoning Modes



SEBASTIAN RASCHKA, PHD

JUL 18, 2026



349



38



32

Share

It has been almost two years since OpenAI released o1, a model that popularized the idea of LLM-based reasoning models. DeepSeek-R1 followed about four months later together with details of a reinforcement learning with verifiable rewards (RLVR) recipe to train such reasoning models.

Last week, OpenAI released the GPT-5.6 model family. It comes in three sizes, each with roughly five or six reasoning-effort settings.

GPT-5.6 Sol on the Artificial Analysis Coding Agent Index v1.1

Codex harness · Composite average pass@1 across DeepSWE, Terminal-Bench v2, and SWE-Atlas-QnA · Higher is better

Source: artificialanalysis.ai/agents/coding-agents/

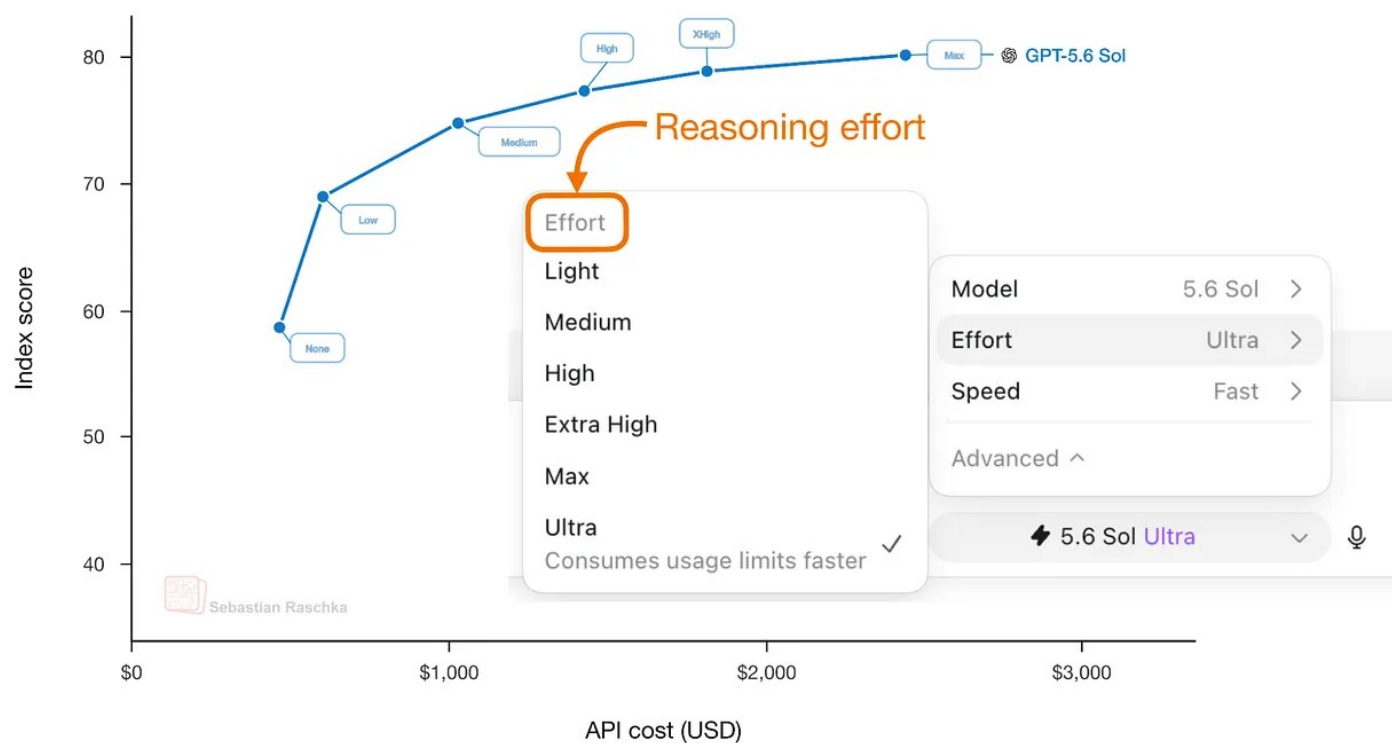


Figure 1: The GPT 5.6 Sol model with different reasoning effort settings. (Benchmark numbers for Ultra are currently not available but should be relatively similar to Max, since it uses a similar effort level but accelerates the work with four subagents.)

So yes, reasoning models are here to stay. They have become a standard part of modern model releases.

In the past, I covered the methodology of reasoning models ([Understanding Reasoning LLMs](#)) as well as relevant research papers ([The State of Reinforcement Learning for LLM Reasoning](#) and [The State of LLM Reasoning Model Inference](#)). And even wrote a whole new 440-page book on how to develop reasoning models, [Build a Reasoning Model \(From Scratch\)](#).



Figure 2: My new [Build A Reasoning Model \(From Scratch\)](#) book. In color!

These resources have focused on turning a conventional LLM into a reasoning model. Now, in this article, I want to focus on and explain how to develop a reasoning model that has multiple effort modes, similar to what's shown in the figure at the beginning.

of this article.

No worries, this article can be read as a standalone article. However, the aforementioned resources may be interesting and useful.

1. A brief definition of reasoning mode

When talking about pretty much any machine learning or AI technique or subfield, one lesson is that we usually shouldn't take technical terms "literally". For example, (artificial) neural network in machine learning and AI doesn't literally work like a biological neural network like the human brain.

Similarly, when talking about "reasoning models", we shouldn't expect that these models literally reason like us humans. In the context of AI and LLM research, "reasoning model" means a model that outputs an intermediate reasoning trace, which is like an intermediate response that works through a question or task step by step.

It's probably easiest to explain this by showing an example.

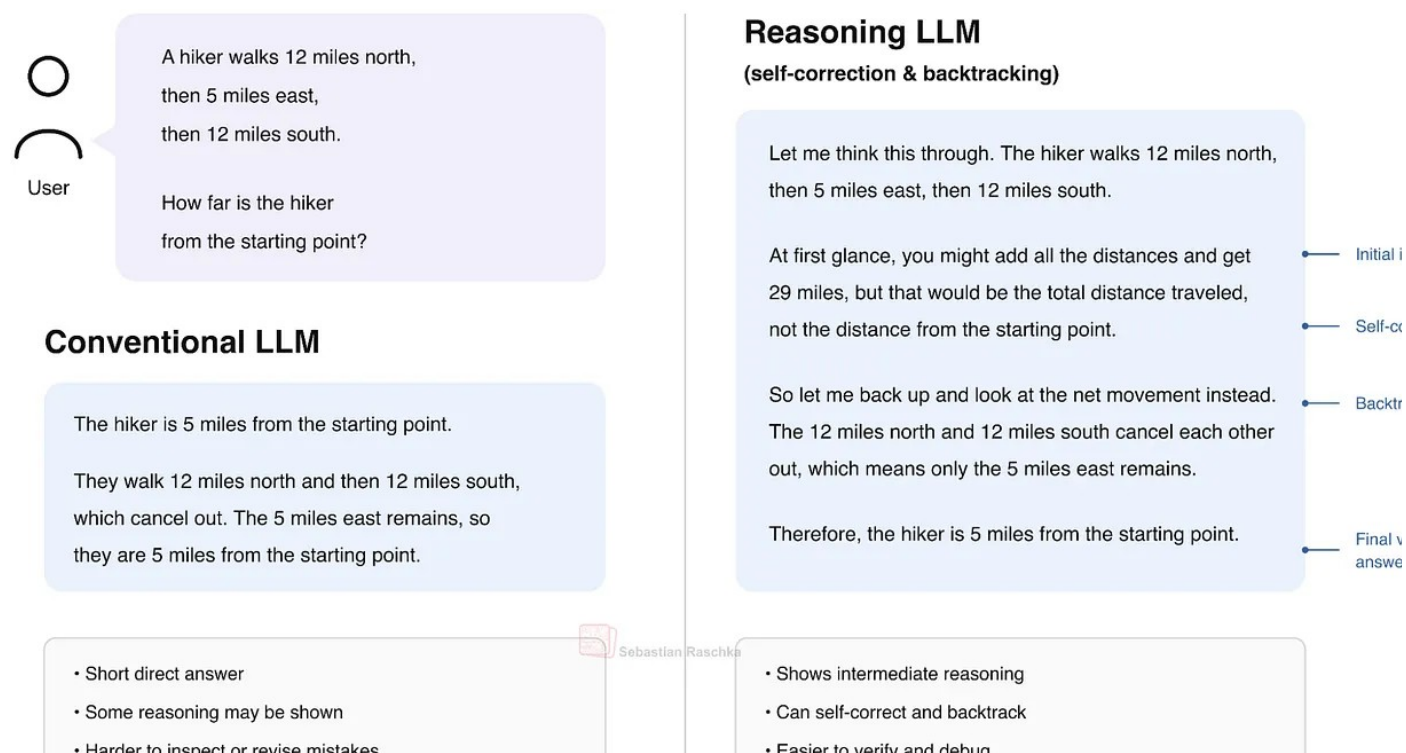




Figure 3: Illustration of a conventional LLM answer (left) and an answer by a reasoning model (right).

2. A brief overview of training and inference scaling reasoning models

There are essentially two ways to improve (reasoning) task performance: training scaling and inference scaling.

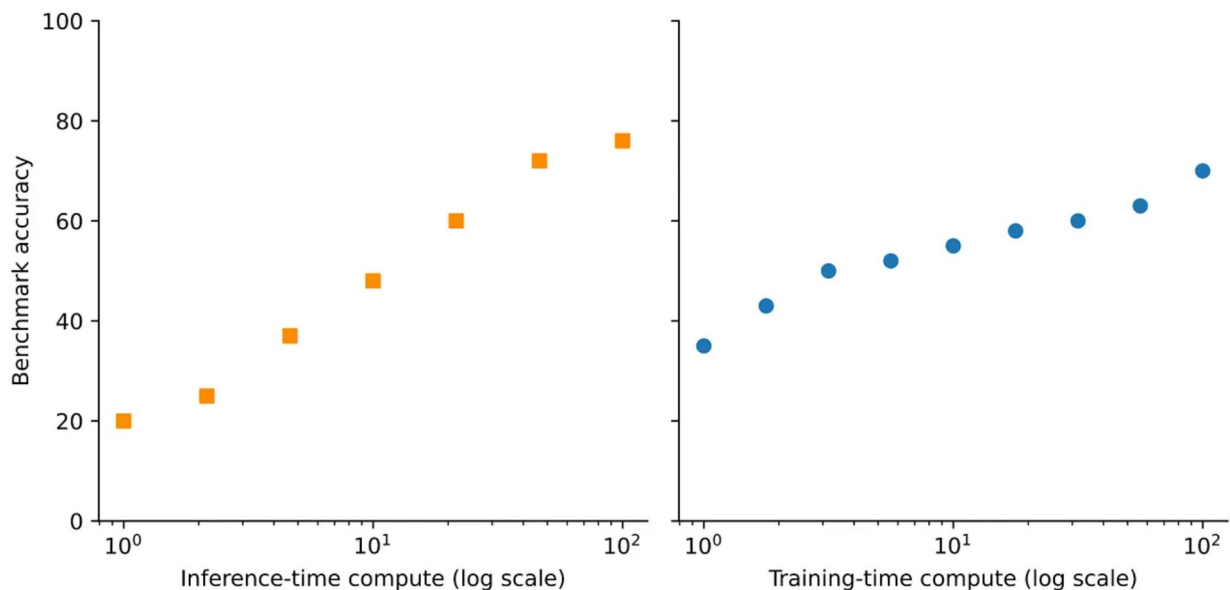


Figure 4: Training and inference-scaling are two ways to improve LLM and reasoning model problem-solving capabilities. Plot based on [Learning to reason with LLMs](#)

Let's briefly talk about training first.

2.1 Training reasoning models

In a nutshell, [DeepSeek-R1](#) proposed training an LLM using reinforcement learning with verifiable rewards (RLVR) to turn it into a reasoning model. RLVR is a technique provide a reward signal (0=incorrect and 1=correct) for verifiable data domains.

These verifiable data domains here are math (we can use a symbolic math checker like SymPy or WolframAlpha to check results) and code (we can use a compiler or unit tests, or integrated platforms like LeetCode) to check for correctness.

What the rewards checked

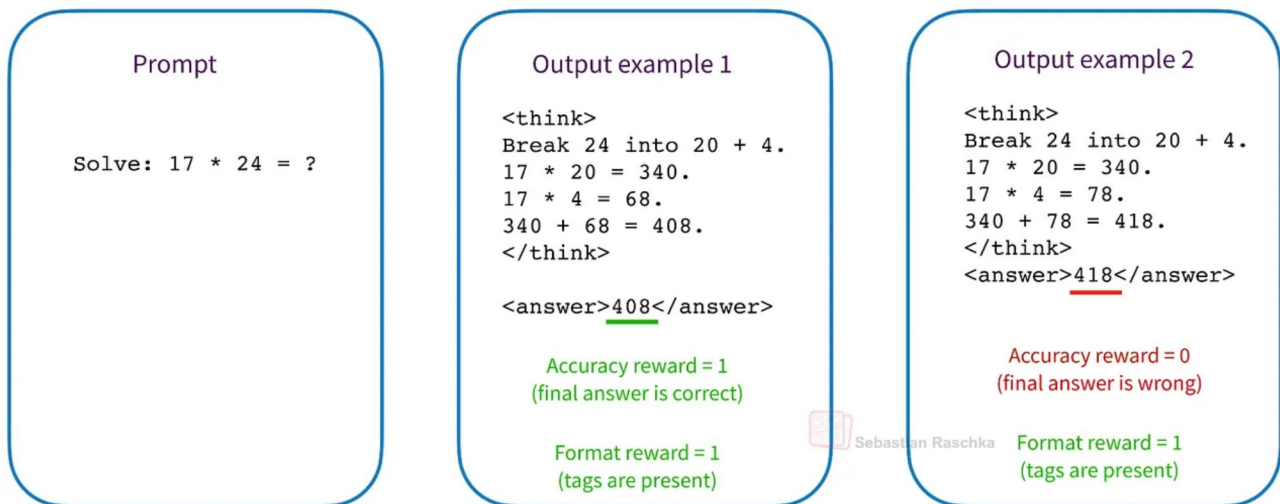
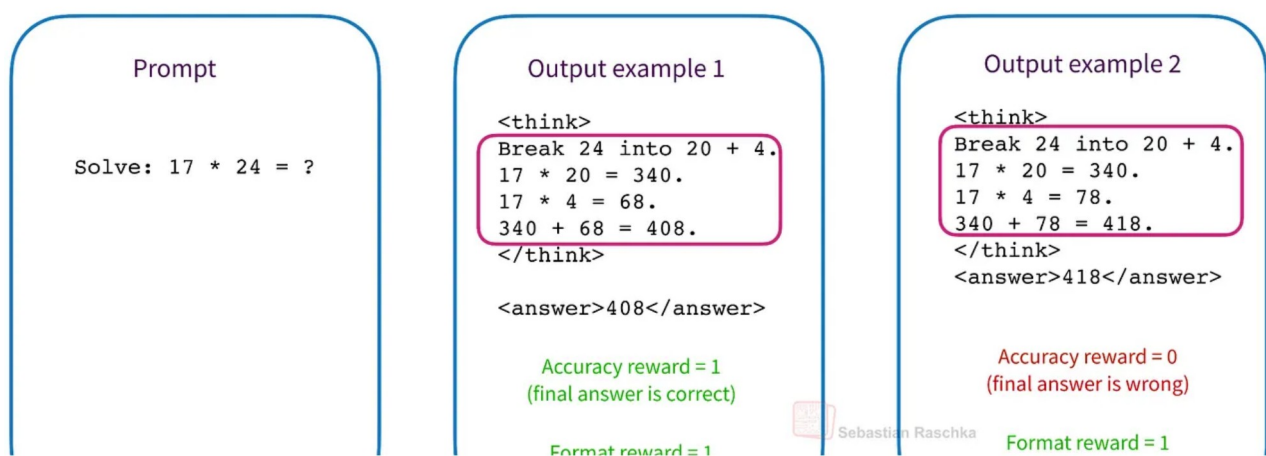


Figure 5: Illustration of accuracy and format rewards during RLVR training.

Notably, the reasoning trace itself was not used for training or updating the model. Although they tried to use this intermediate response information for training, the DeepSeek-R1 paper reported that it wasn't helpful for the model training, so it was ultimately not used. (Whether and how to incorporate intermediate reasoning trace the training signal via process reward models is an active area of research.)

What is ignored



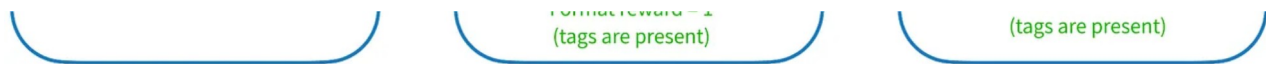


Figure 6: The intermediate reasoning trace is ignored during RLVR; only the final answer and response format determine the reward.

2.2 “Aha” moments

Anyway, just training on the output rewards alone, as Figure 7 shows, turned out to be sufficient for the model to learn how to reason through a problem, meaning that it would learn to write intermediate explanations, backtrack, and self-correct itself. The moments when the model realizes that it made a mistake and self-corrects itself are called “Aha” moments.

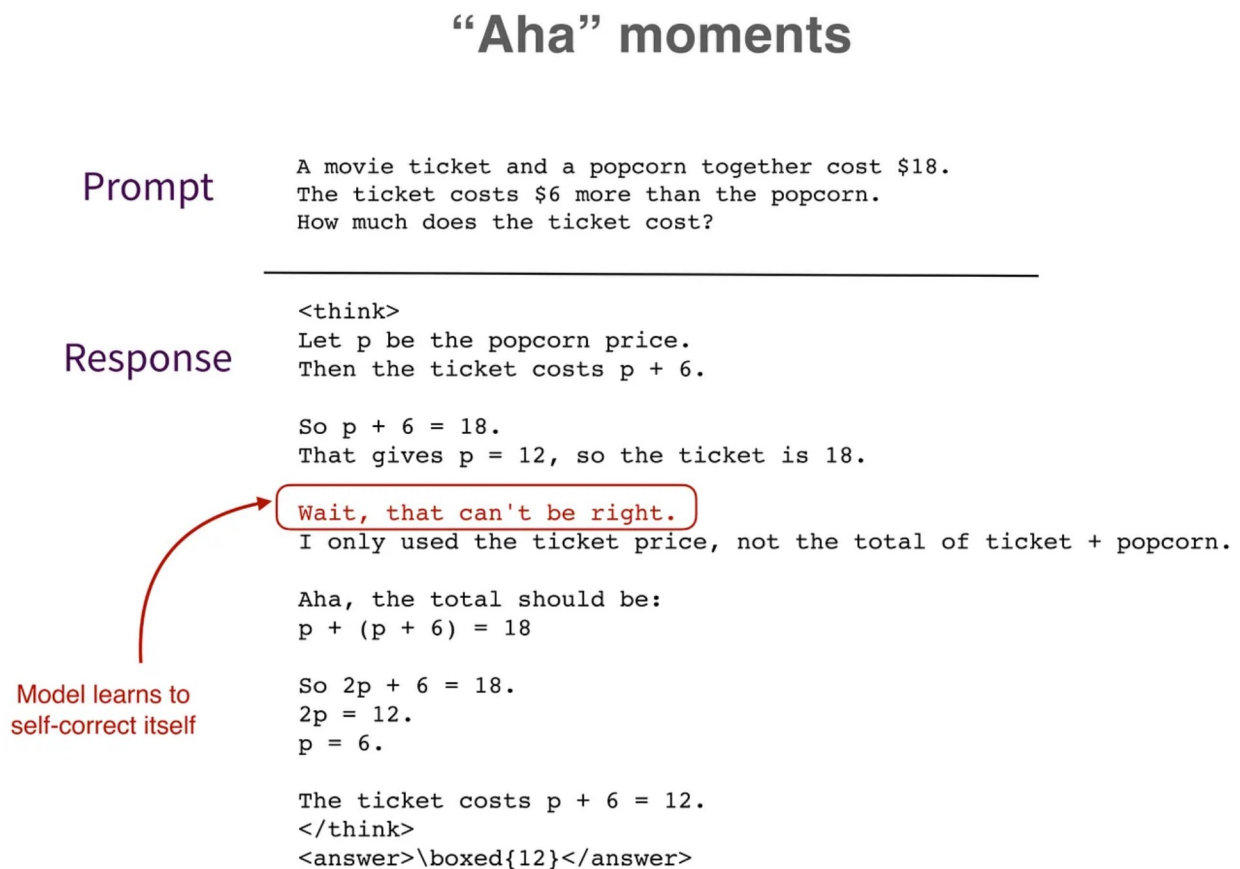


Figure 7: An example of an aha moment, where a reasoning model notices an error in its intermediate reasoning and corrects it before producing the final answer.

By the way, while DeepSeek-R1 is inarguably the more popular paper, and the paper that created excitement around reinforcement learning with verifiable rewards and

development of reasoning models, there is another paper, [Kimi K1.5](#), published on exactly the same day on arXiv (22 Jan 2025). Also, the term RLVR was already coined two months earlier in [Tülu 3: Pushing Frontiers in Open Language Model Post-Train](#)

One reason why the DeepSeek R1 is ultimately the more popular paper is that it demonstrated that reasoning behavior can be achieved with pure reinforcement learning (RL).

Pure RL on the base model, no SFT

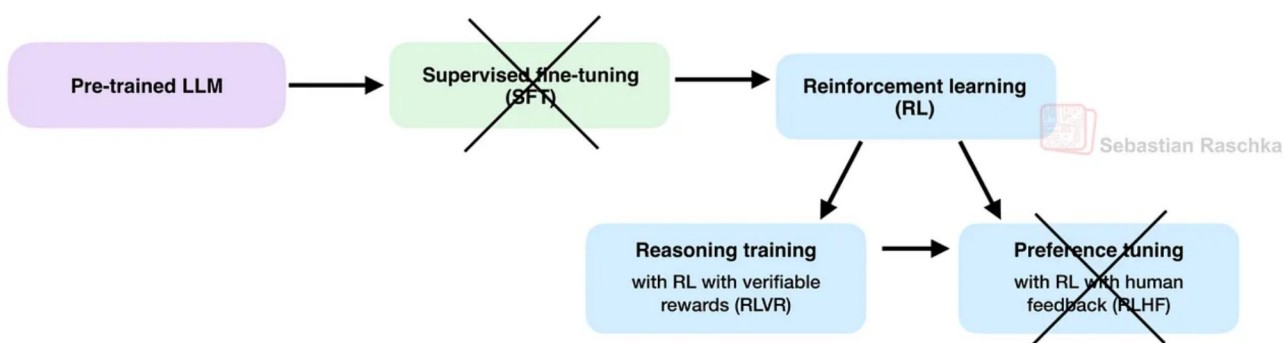
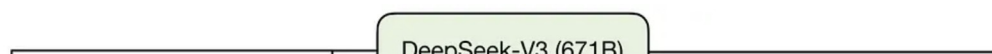


Figure 8: DeepSeek-R1-Zero applies RLVR directly to the pretrained base model without supervised fine-tuning.

For instance, Tülu 3 and Kimi K1.5 applied reinforcement learning on top of a supervised fine-tuned (SFT) model. The DeepSeek-R1 model was also trained from a SFT checkpoint of the DeepSeek-V3 base model, and it included a DeepSeek-R1-Zero variant trained with pure RLVR. R1 Zero is a weaker model than R1, but it showed that RLVR is sufficient for teaching the model to generate and use reasoning traces.

While R1-Zero was more of a proof-of-concept model, note that the full DeepSeek-reasoning model training pipeline is usually multi-stage and a bit more complicated than mentioned above.



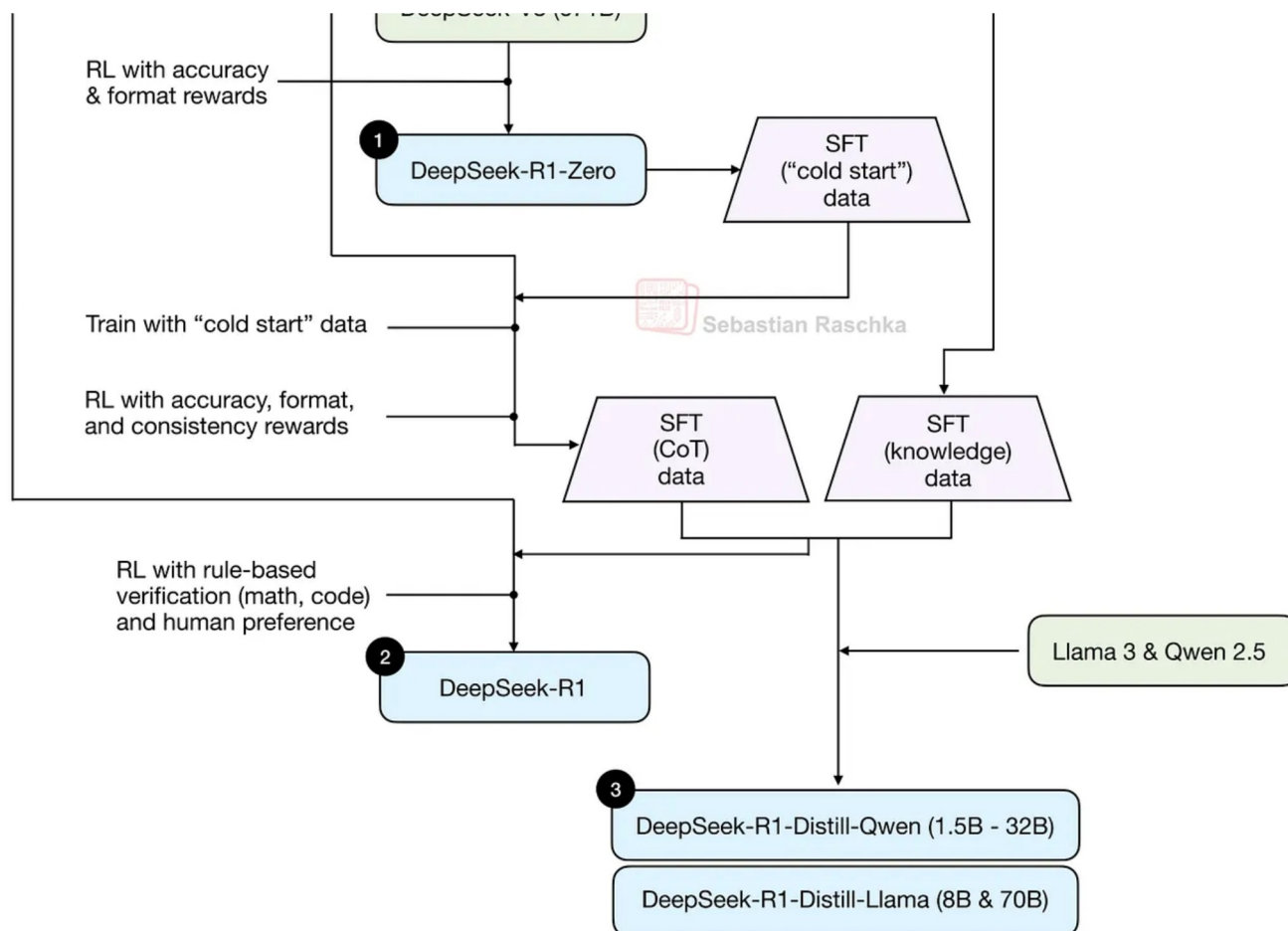


Figure 9: More detailed reasoning model training pipeline. This one depicts the various DeepSeek-R1 models. For more details, see my other article: [Understanding Reasoning LLMs](#)

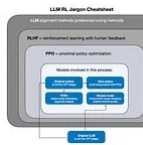
By the way, most of today's LLMs are effectively reasoning models, meaning they have been trained in a similar fashion to DeepSeek-R1 using a form of RLVR.

2.3 Inference scaling in a nutshell

Next to improving reasoning behavior through training, another lever for improving model performance is inference compute scaling. In short, this means that we are spending more compute after training the model, during usage, to get better answers.

This is a whole topic by itself, and you could read through my [The State of LLM](#)

Reasoning Model Inference for a more detailed rundown:



The State of Reinforcement Learning for LLM Reasoning

SEBASTIAN RASCHKA, PHD • APRIL 19, 2025

[Read full story →](#)

I will try to summarize what's most essential to mention as background info below.

First, training a model with RLVR is already implicitly leading to a form of inference scaling, since reasoning models usually output more tokens during inference compared to conventional LLMs, and that means we are spending more compute during inference.

Second, we can further adjust this output length via reasoning effort levels, but more on that later.

Third, there are many additional inference scaling techniques. A popular one is self-consistency, which is often implemented as a form of majority voting where the model is queried multiple times, and the final answer is selected via majority vote.

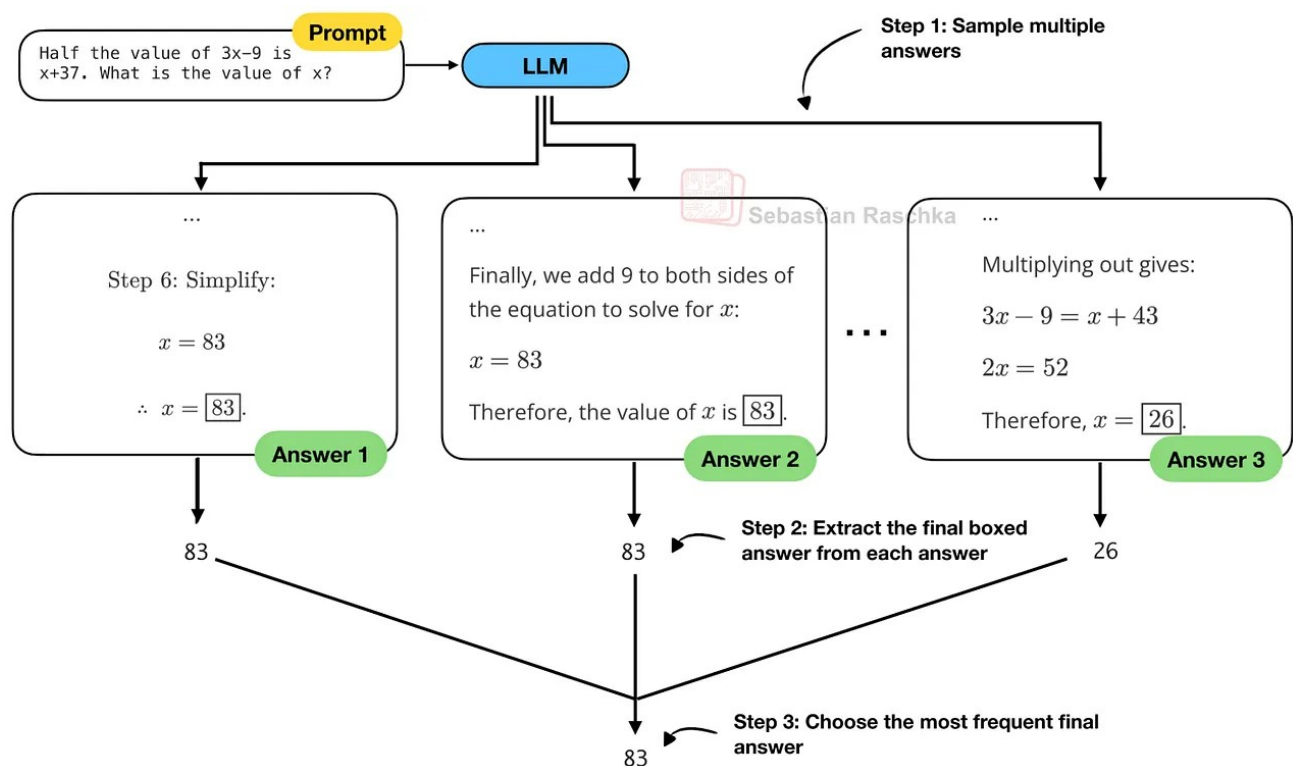


Figure 10: An example of self-consistency, a popular inference scaling technique.

This can be applied to conventional LLMs as well as reasoning models. Also, this method can be used on demand and in addition to reasoning training. A good example of that is DeepSeekMath-V2, where the researchers applied extreme inference-scaling on top of a reasoning model (specialized for math) to achieve state-of-the-art performance on challenging math olympiad-type problems.

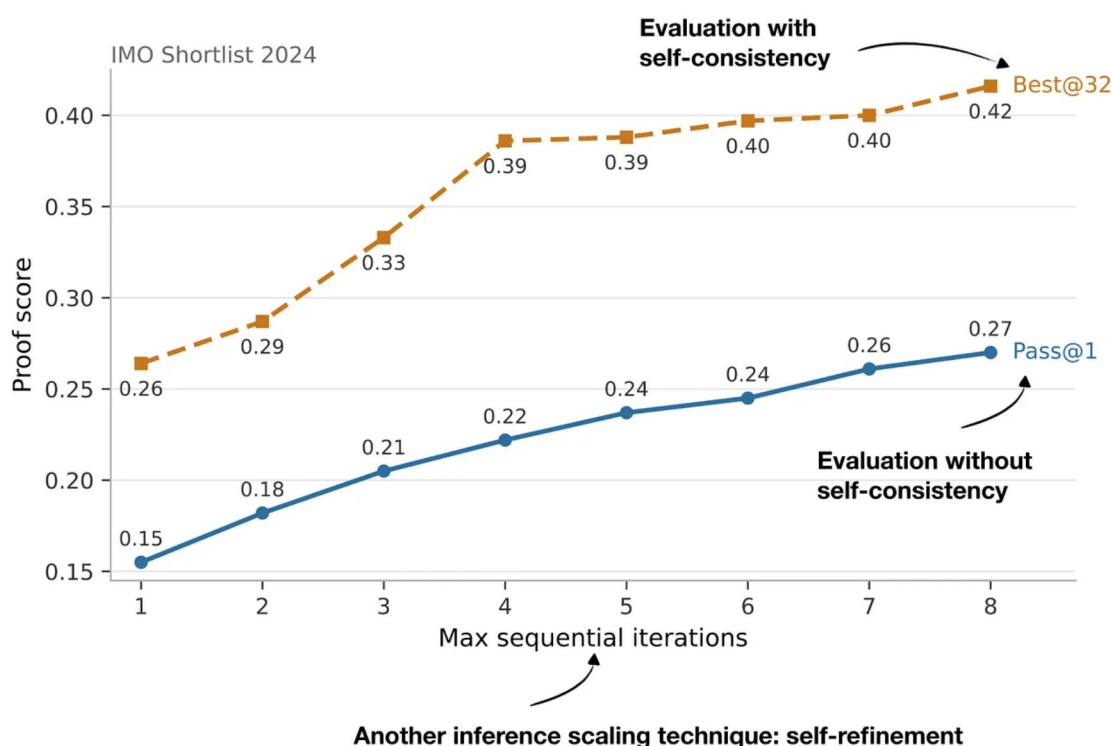
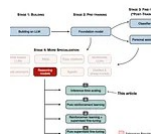


Figure 11: Two types of inference scaling (self-consistency and self-refinement) used together to improve math performance. Figure adapted from [DeepSeekMath-V2: Towards Self-Verifiable Mathematical Reasoning](#)

But again, I will refer to my other article, [The State of LLM Reasoning Model Inference](#) for an overview of other techniques:



The State of LLM Reasoning Model Inference

SEBASTIAN RASCHKA, PHD • MARCH 8, 2025

[Read full story →](#)

3. Think tokens

You may have seen the `<think></think>` tokens in the earlier “Aha moments” figure. I also included the corresponding figure below so you don’t have to scroll all the way up.

Prompt	A movie ticket and a popcorn together cost \$18. The ticket costs \$6 more than the popcorn. How much does the ticket cost?
Response	<pre> <think> Let p be the popcorn price. Then the ticket costs p + 6. So p + 6 = 18. That gives p = 12, so the ticket is 18. Wait, that can't be right. I only used the ticket price, not the total of ticket + popcorn. Aha, the total should be: p + (p + 6) = 18 So 2p + 6 = 18. 2p = 12. p = 6. The ticket costs p + 6 = 12. </think> <answer>\boxed{12}</answer> </pre>

Figure 12: Common formatting tokens in reasoning models.

These `<think>` and `</think>` tags are cosmetic with respect to reasoning ability. They do not make the model reason, and they are not required to achieve good reasoning performance. One could train the same model without these delimiters and likely reach similar benchmark performance.

The purpose of these `<think>` tags or tokens is mainly to mark where the reasoning trace begins and ends so that the training pipeline or user interface can separate it from the final answer and optionally hide it from the user. (UIs like ChatGPT or Code

usually do this.)

The point here is that the `<think>` tokens are not giving the model the ability to “think” or reason or reason better. One could train the same models without such `<think>` tokens and reach similar benchmark performance.

There is also nothing special about the literal strings `<think>` and `</think>`. Another pair of delimiters could serve the same purpose.

By the way, the way this is implemented is typically by adding a formatting reward during the RLVR stage. So instead of just rewarding the model based on answer correctness, one would provide additional reward for the use of `<think>` tokens, which in turn encourages the model to use those.

In DeepSeek-R1, for example, the overall reward was calculated as

$$R_{\text{total}} = R_{\text{accuracy}} + R_{\text{format}}$$

where the format reward was a simple rule-based check that encouraged the model to place its reasoning inside:

```
<think>
```

```
reasoning trace
```

```
</think>.
```

4. Reasoning mode on and off switches

The first generation of reasoning models was dedicated reasoning models. With that meant that there was a DeepSeek-V3 base model and a separate DeepSeek-R1 reasoning model.

No matter what the prompt is, R1 generally outputs very verbose responses using lots of tokens, even for simple prompts. It also lacks a built-in option to turn off the reasoning mode.

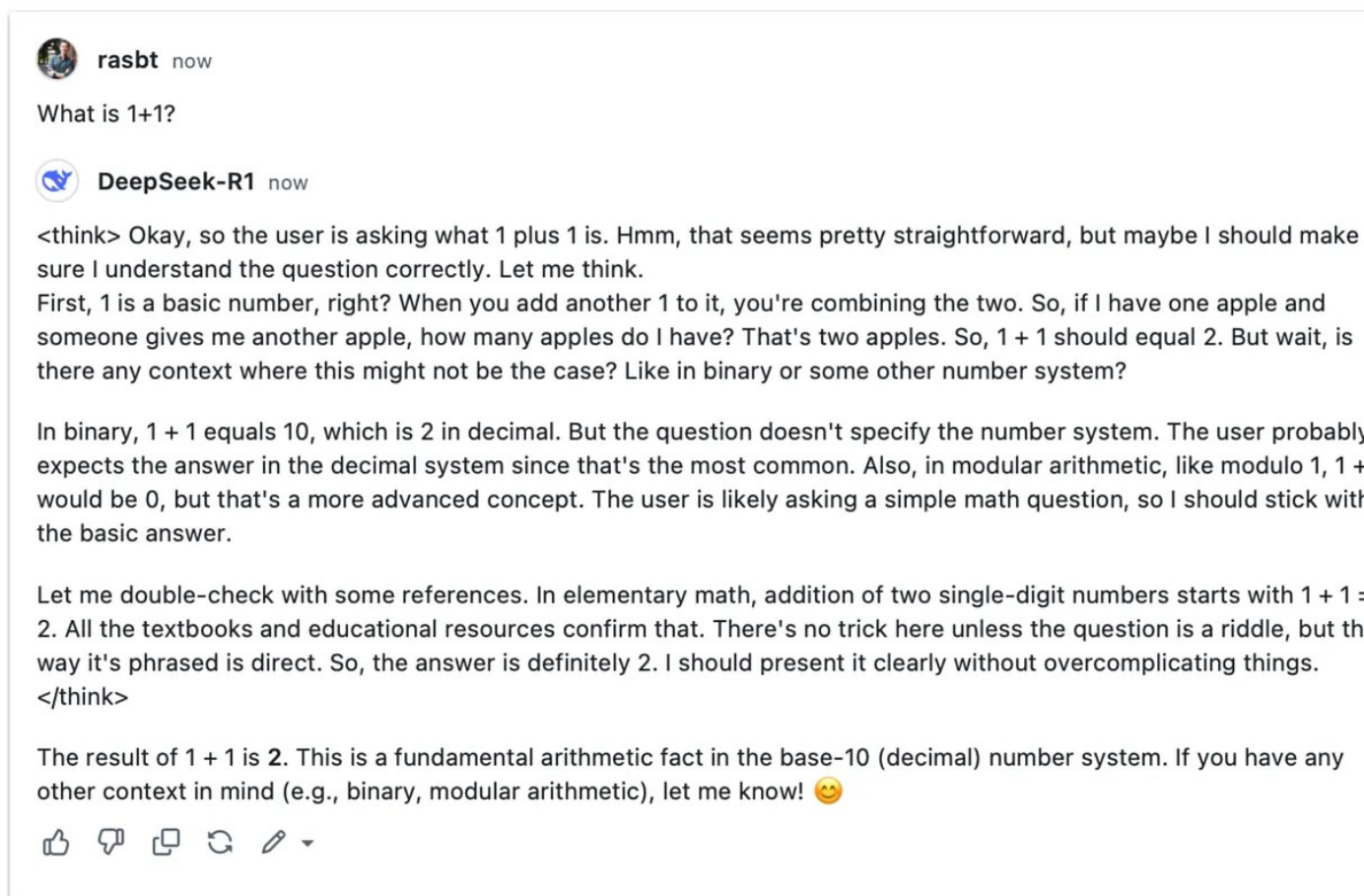


Figure 13: Reasoning models are very verbose, even for the simplest prompts.

Later models, like Qwen3 and others, experimented with hybrid approaches, where same model can behave like a regular instruction fine-tuned model or a reasoning model on demand.

Note: Some model developers call this “thinking mode,” while others call it “reasoning mode.” Both terms refer to the same behavior.

In Qwen3, this is handled via the tokenizer using `enable_thinking=True` or `enable_thinking=False`. Under the hood, setting `enable_thinking=False` essentially adds an empty `<think></think>` section to the beginning of the assistant's response.

response to turn off Qwen3's reasoning ("thinking") mode.

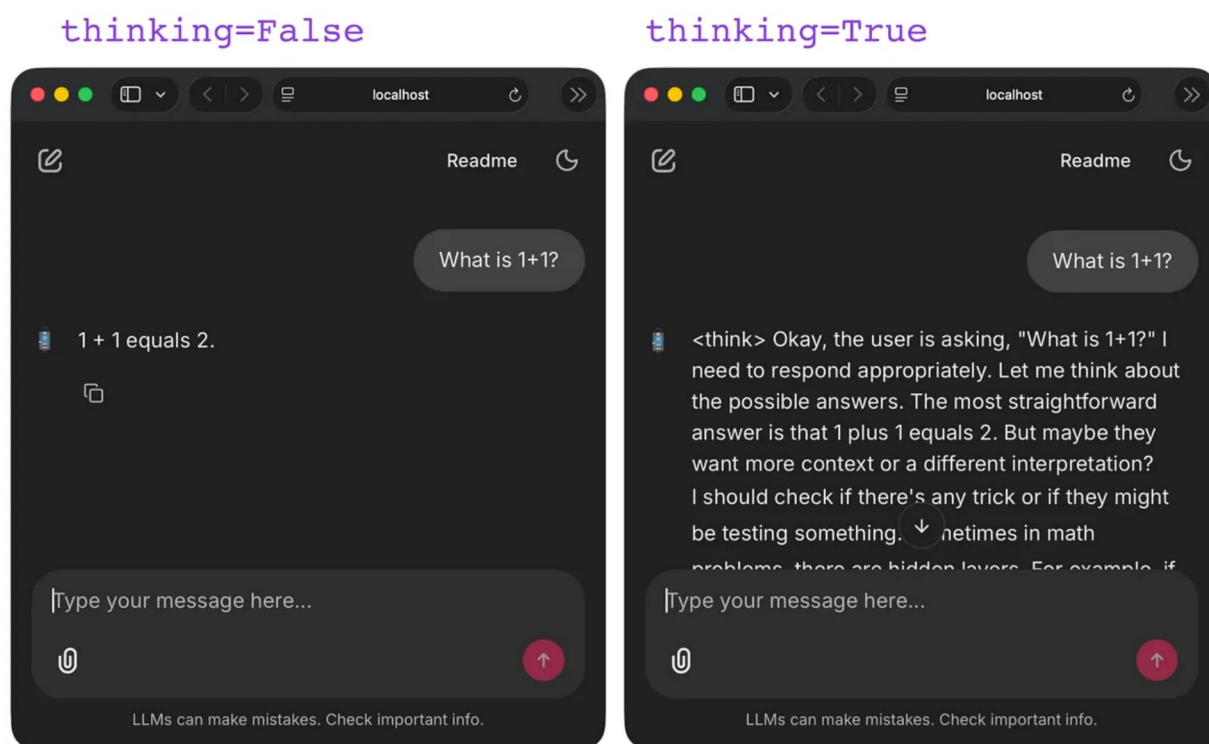


Figure 14: Response of Qwen3 0.6B reasoning model with `thinking=False` and `thinking=True`. (The empty `<think></think>` tags are hidden in the interface on the left as they are part of the modified input prompt, not the generated answer.)

How is this implemented during training, such that the model supports this toggle during inference time, as shown in the figure above?

In short, as explained in the [Qwen3 technical report](#), this on/off behavior is introduced primarily through supervised fine-tuning (SFT) and then reinforced during general F in their largest flagship models.

For instance, after the initial reasoning model is trained via long-chain-of-thought S and reasoning RL, they add a "Thinking Mode Fusion" stage. During this additional stage, the model sees both thinking and non-thinking examples:

- `/think: <think>{reasoning}</think>{answer}`
- `/no_think: <think></think>{answer}`

Thinking is the default behavior, so `/think` can also be omitted. The subsequent gen RL stage further reinforces this mode and format following.

These `/think` and `/no_think` flags are a “soft” switch. However, the `enable_thinking=False` setting mentioned earlier, which force-adds the empty `<think></think>` in the False case, acts then as a “hard” switch.

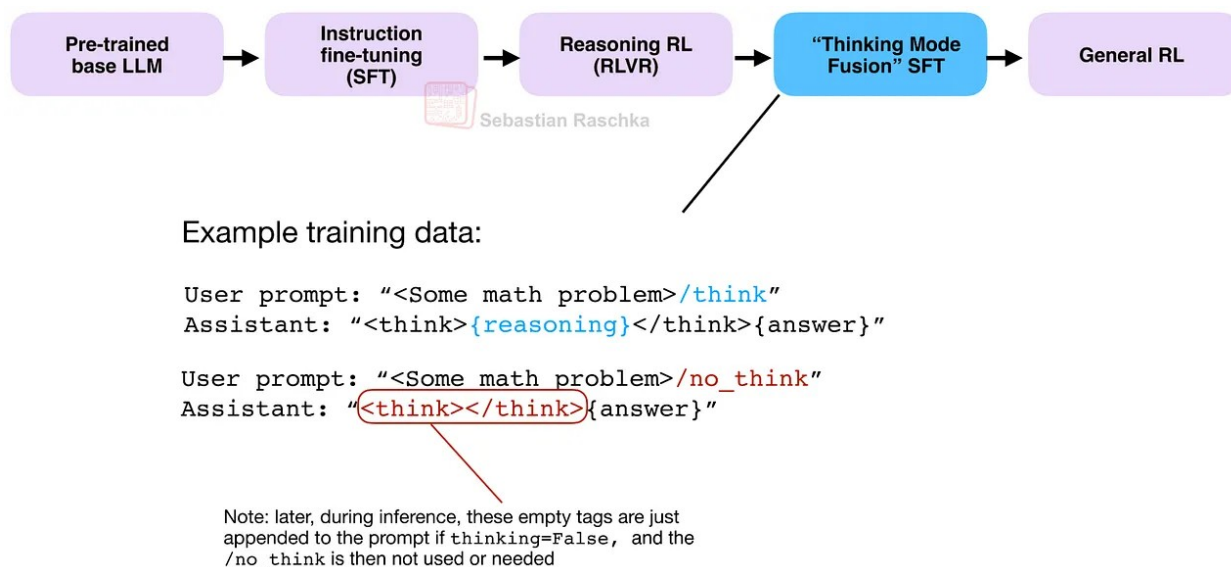


Figure 15: "Thinking Mode Fusion" in Qwen3's training pipeline to enable the reasoning mode on and off switch.

In other words, the tokenizer does not add `/no_think` to the query. It directly fills in the empty `<think></think>` section at the beginning of the assistant response. The model only sees the resulting tokens and continues directly with the answer.

Anyway, this on-and-off toggle is essentially a simplified version of the reasoning effort levels in GPT-5.6 and others, which I cover in the next section.

5. How “reasoning effort” settings work

In this section, I want to provide a brief overview of how the different reasoning effort toggles may be implemented, which have been introduced in models like GPT-5 and

are present in pretty much any flagship model today.

Concretely, at the beginning of this article, I showed a figure from the Codex GPT 5. interface that lets users select multiple reasoning “effort” settings.

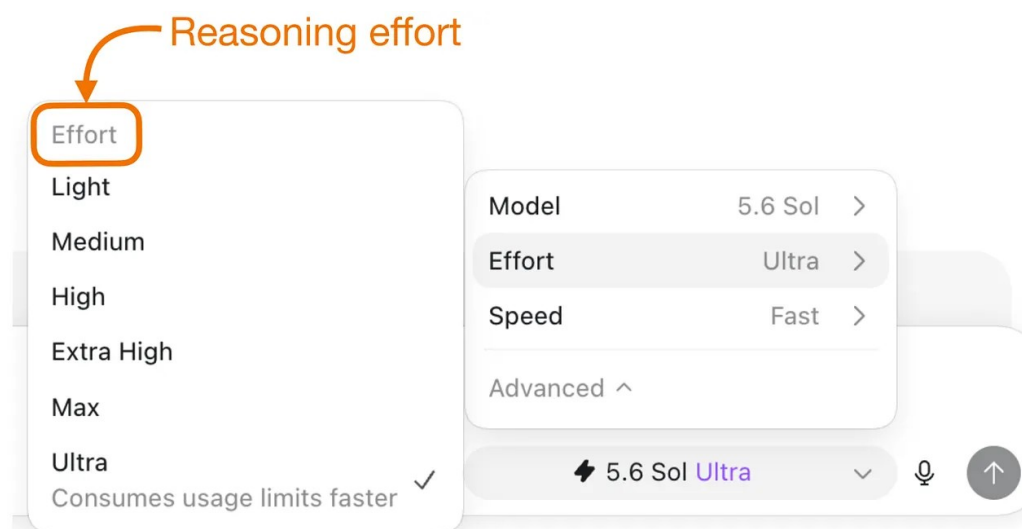


Figure 16: GPT-5.6 exposes six reasoning effort settings, ranging from Light to Ultra.

The following subsection will illustrate how these settings may be implemented. Then, in the next section, I will go over some of the more interesting research papers related to this topic.

5.1 Reasoning effort and response length and quality

Unfortunately, the implementation details of their effort settings are not shared by OpenAI, but there is some evidence out there that can be used for educated guesses.

For instance, via their open-source gpt-oss models from last year (I wrote about them in [From GPT-2 to gpt-oss: Analyzing the Architectural Advances](#)), we know that OpenAI allows us to toggle the reasoning effort setting via the system prompt (“Reasoning effort: low/medium/high”) that is prepended to each prompt.

User-facing input

Simplified prompt received by gpt-oss



Figure 17: The gpt-oss chat template inserts the selected reasoning effort into the system message before sending the prompt to the same model.

As expected, the reasoning effort directly affects the response length and accuracy, shown below.

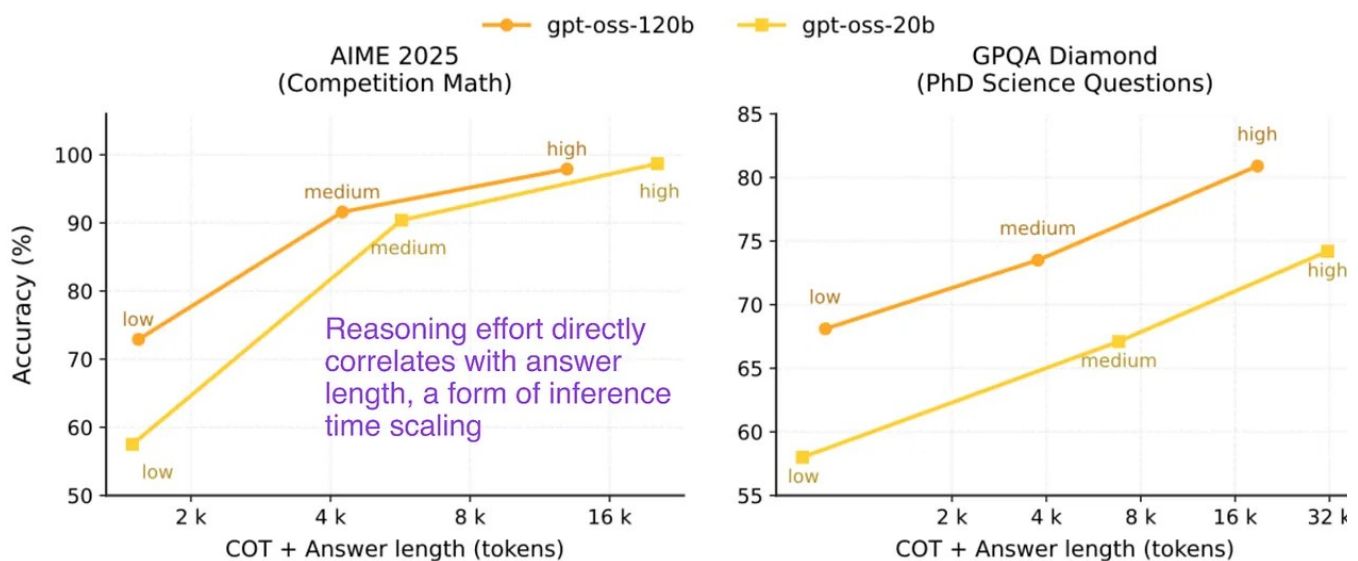


Figure 18: Response length and quality of gpt-oss models under different reasoning efforts (annotated figure from the [model card](#))

Presumably, their GPT 5 models, including the recent GPT 5.6 models, use a similar approach.

By the way, note how different effort settings scale the response length in the figure above. The effort level seems directly correlated to token usage, which in turn seem correlated to accuracy. It might be possible to come up with effort settings beyond

“high” one, but I assume performance would saturate at some point. This saturation can be seen more clearly for the GPT 5.6 Sol model, which also shows that increasing reasoning budgets can become uneconomical at some point.

Artificial Analysis Coding Agent Index v1.1

Index score based on composite average pass@1 across DeepSWE, Terminal-Bench v2, and SWE-Atlas-QnA · Higher is better · Source: artificialanalysis.ai/agents/coding-agents/

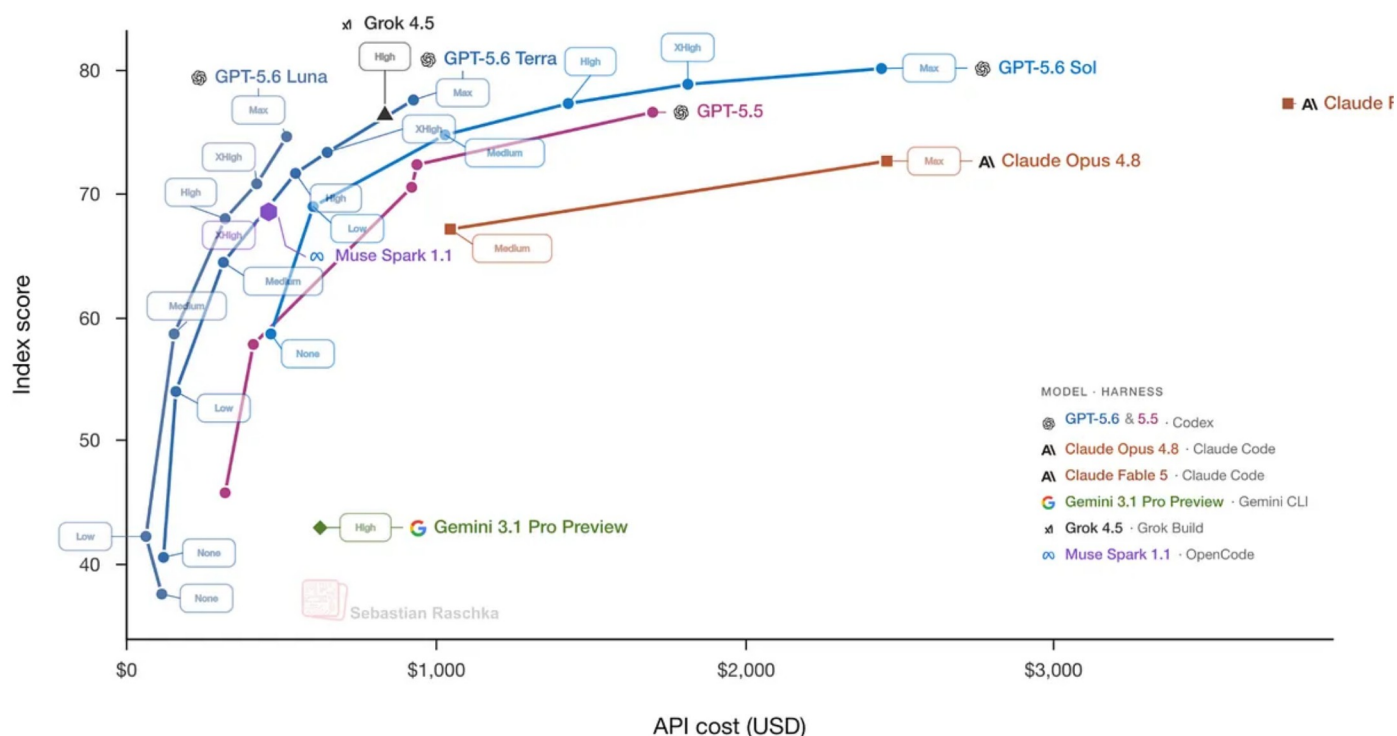
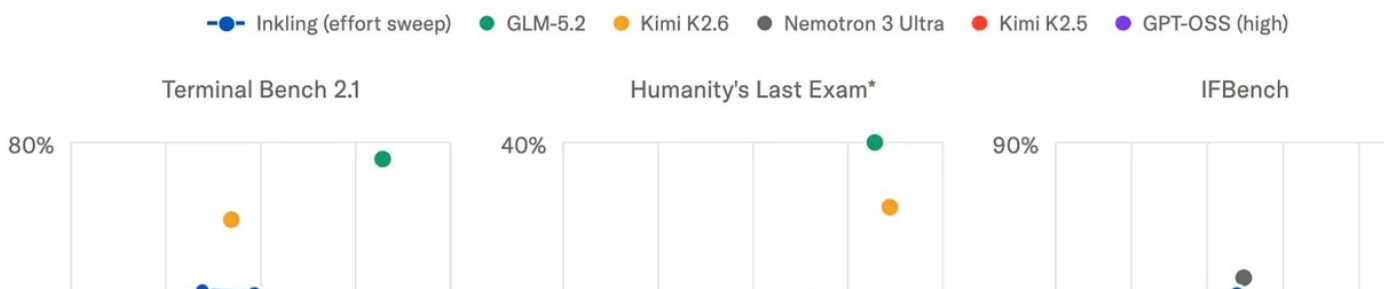


Figure 19: Reasoning effort increases both API cost and coding-agent performance, with diminishing returns at the highest GPT-5.6 settings. Figure based on the Artificial Analysis Coding Agent Index v1.1.

Another good, very recent data point that shows the relationship between reasoning effort, token usage, and benchmark performance is this week’s new [open-weight Inking release](#) by Thinking Machine Labs.



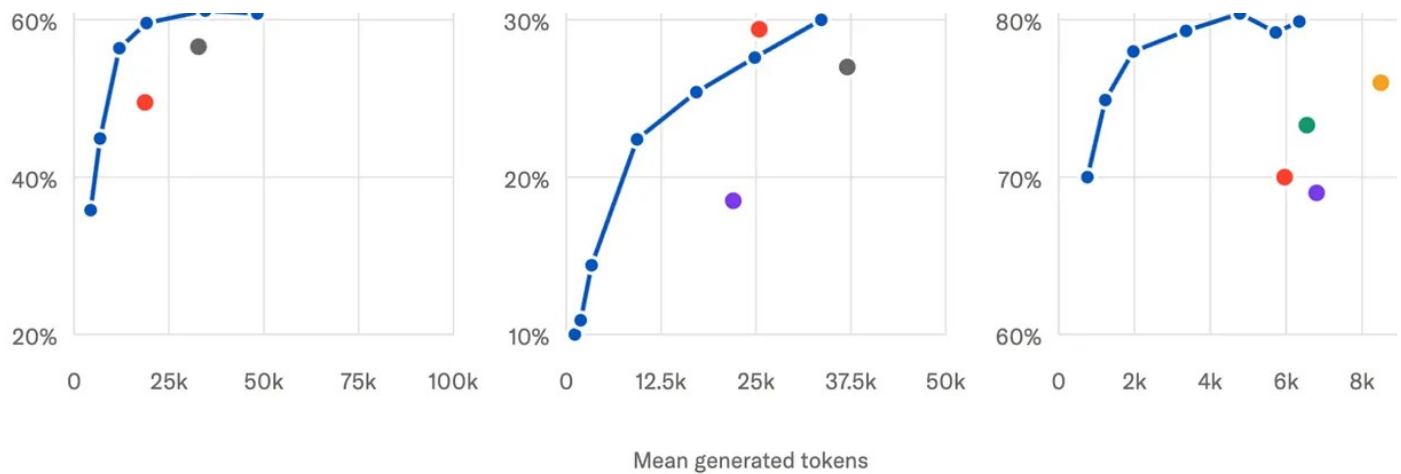


Figure 20: Increasing the Inking effort level generally increases generated tokens and benchmark performance, with diminishing or uneven gains at higher effort.
Figure from the [Inking announcement blog](#).

As discussed in this section, during inference, the reasoning effort level can simply be controlled via a system prompt. (The ChatGPT UI presumably simply maps the menu choice to a system prompt.) However, this would not work for an arbitrary model and requires certain modifications to the training pipeline, which will be discussed next.

5.2 Possible effort level implementations

While the training details are not public, neither for GPT 5.6 nor the open-source gpt-oss models, typically, the reasoning effort label is included in prompts during post-training.

There are typically two ways to implement this.

First, we can implement it as part of the RLVR process and apply a different length penalty when different system prompts are used. For example, a high length penalty when “Reasoning effort: low” and a mild or no penalty when “Reasoning effort: high”.

Second, we can fine-tune the model after RLVR to follow different effort instructions via supervised fine-tuning (SFT).

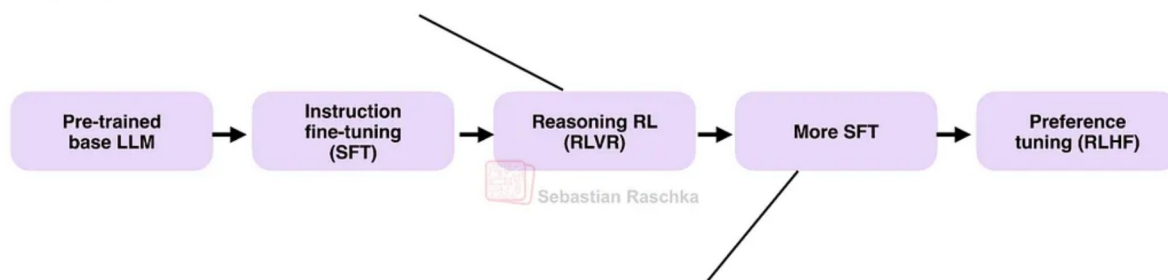
For instance, after the core RLVR stage and during SFT, the prompts in the training

dataset are paired with target responses that exhibit the desired amount of reasoning (The targets may be written by humans, generated by another model, or generated and then filtered.)

Example training data:

System prompt: "Reasoning: **low**" → **High** length penalty in RLVR/GRPO training
 User prompt: "<Some math problem>"

System prompt: "Reasoning: **high**" → **Low** length penalty in RLVR/GRPO training
 User prompt: "<Some math problem>"



Example training data:

System prompt: "Reasoning: **low**"
 User prompt: "<Some math problem>"
 Assistant: "<**Short** reasoning trace>"

System prompt: "Reasoning: **high**"
 User prompt: "<Same math problem as above>"
 Assistant: "<**Longer, more detailed** reasoning trace>"

Figure 21: Illustration of effort-conditioned RLVR and SFT. (This is a possible implementation, not a confirmed description of OpenAI's training pipeline.)

During this SFT stage, the model learns the association between the effort label and the target reasoning length directly from the training examples. An RL-based implementation would instead place the effort labels and budget-aware reward inside the RLVR stage. The two approaches could also be combined, which I suspect was done for both gpt-oss and GPT 5.6 (note that effort settings in GPT 5.6 are likely just changing the system prompt for a given user query).

5.3 Inkling case study

The just-released Inkling technical report gives a small but somewhat concrete

example of effort-level training.

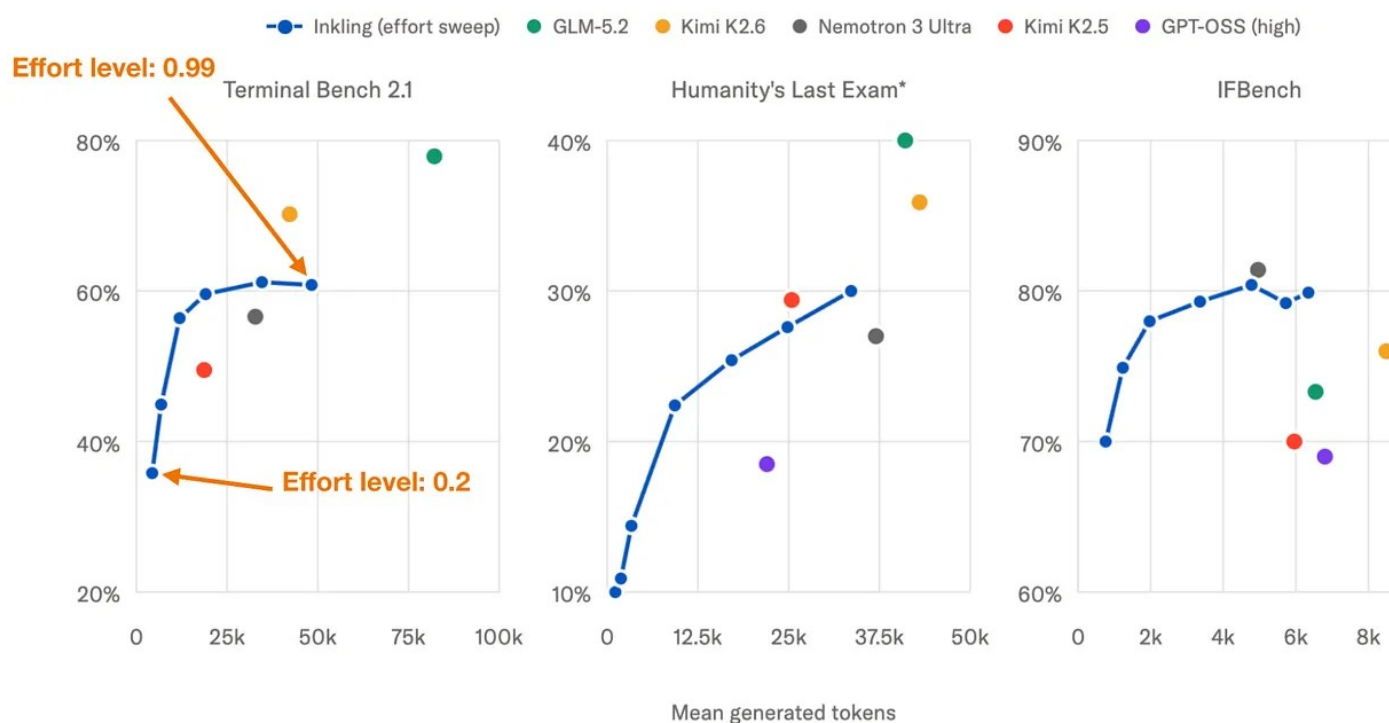


Figure 22: Inking sweeps a continuous effort value between 0.2 and 0.99; higher effort generally produces longer responses and higher benchmark scores.

During large-scale RL, they did two things for each sample:

1. Specified the desired effort level in the system message.
2. Adjusted the cost assigned to each generated token.

Conceptually, the reward likely looked something like this:

$$R(e) = R_{\text{task}} - \lambda(e)N_{\text{tokens}}$$

Here, e is the requested effort level and $\lambda(e)$ controls the token penalty.

- Low effort uses a larger per-token cost, encouraging shorter reasoning traces.
- High effort uses a smaller per-token cost, allowing the model to spend more tokens.

Then, at inference time, Inkling receives a system message such as Thinking effort level: 0.8, and adjusts its token usage accordingly. The difference between Inkling and models such as gpt-oss and GPT-5.6 is that the effort label is a continuous number between 0 and 1 instead of ordinal labels such as low, medium, and high.

This places Inkling's effort-level conditioning primarily in the Reasoning RL stage, not only in the later SFT stage.

They do not disclose the exact reward formula, token-cost coefficients, or whether effort conditioning was also included in SFT, though.

5.4 A short note about inference scaling versus training scaling

Before moving on to the reasoning effort papers, I want to briefly connect this section back to the earlier "2.3 Inference scaling in a nutshell" section.

Earlier, I separated scaling into training compute scaling and inference-time scaling. The GPT-5.6 interface provides a nice way to illustrate the difference, as shown below.

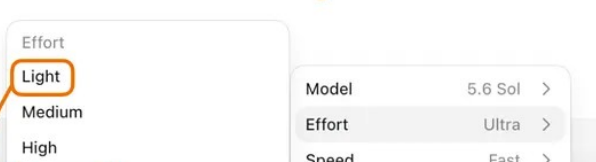
On the left, selecting Luna, Terra, or Sol changes the model itself. As a rough analog, this corresponds to training compute scaling. These are separate trained models. At a fixed training recipe and dataset size, a larger model requires more training compute. It also generally requires more compute per generated token.

On the right, we keep the model fixed and only change the reasoning effort. This is inference-time scaling. The model weights stay the same, but the model is allowed to spend fewer or more tokens working on the answer.

Training compute scaling



Inference-time scaling



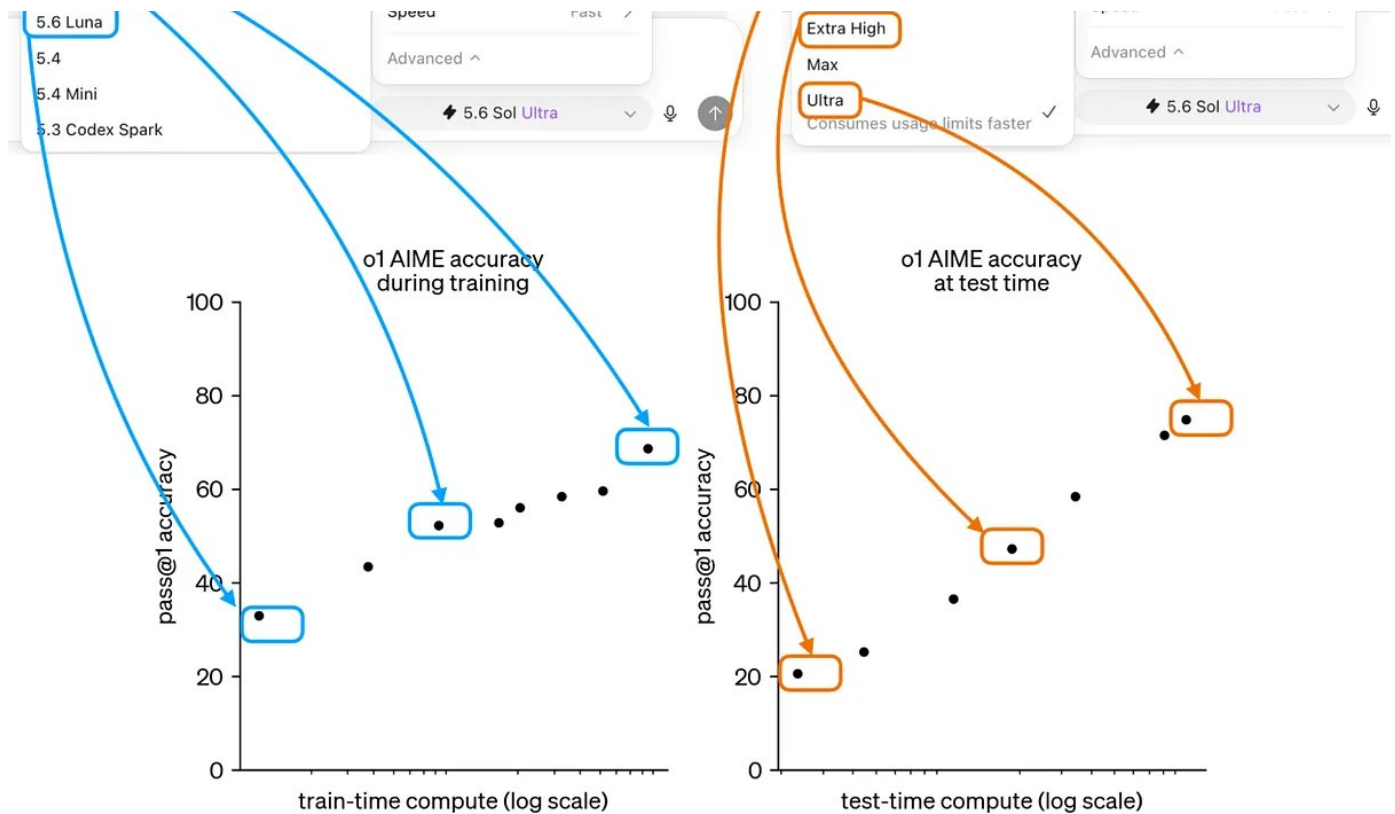


Figure 23: The model selection and reasoning effort menus correspond to two different scaling axes. Selecting Luna, Terra, or Sol changes the model, whereas changing the reasoning effort adjusts the inference-time compute for a fixed model.

One small terminology caveat is that selecting a different model from the menu is *not* training scaling at that moment. The training has already happened. It is better to think of the model menu as selecting among models that were produced at different training scales.

The Artificial Analysis results below show how these two axes interact in practice.

Each blue curve corresponds to one model, Luna, Terra, or Sol. Moving along a curve by increasing the reasoning effort is inference scaling. Moving from one model curve to another corresponds to model scaling, which I use here as a practical proxy for training scaling.

As expected, both approaches can improve the benchmark score, but they also increase the cost. More interestingly, the curves overlap. For instance, a smaller model at a higher reasoning effort can sometimes reach a similar score as a larger model at a lower reasoning effort.

lower reasoning effort.

GPT-5.6 models on the Artificial Analysis Coding Agent Index v1.1

Sol, Terra, and Luna · Codex harness · Composite average pass@1 across DeepSWE, Terminal-Bench v2, and SWE-Atlas-QnA · Higher is better
Source: artificialanalysis.ai/agents/coding-agents/

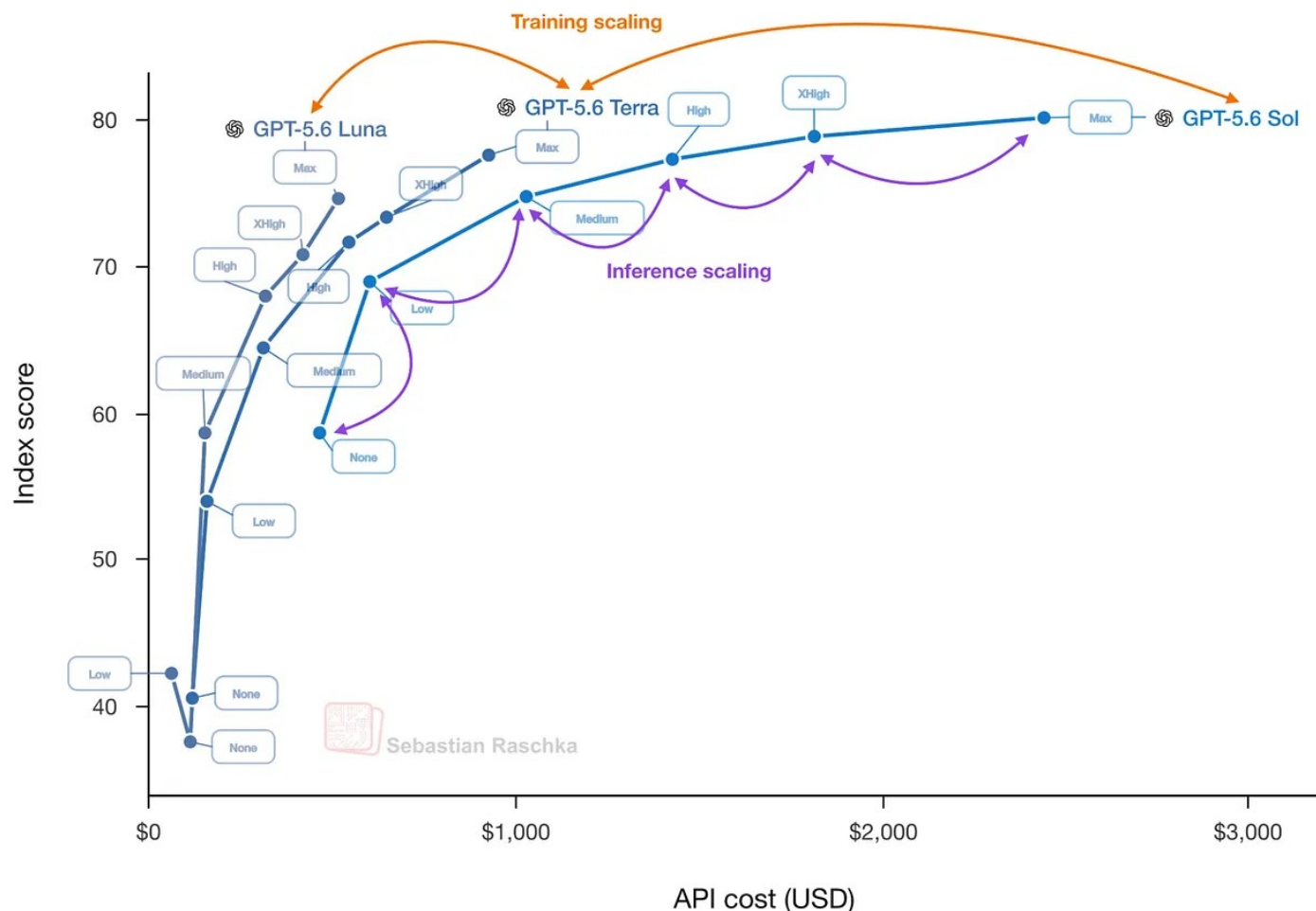


Figure 24: Training scaling and inference scaling for the GPT-5.6 model family on the Artificial Analysis Coding Agent Index. Moving along each model curve corresponds to increasing the reasoning effort. Moving across the Luna, Terra, and Sol curves corresponds to selecting a different model.

By the way, the x-axis in this figure shows API cost rather than raw compute. The API cost is a useful practical measure, but it also depends on the provider's pricing and number of generated tokens. Also, the exact shape of these curves is benchmark-specific.

So, the model size and reasoning effort form two separate knobs. We can use a larg

model, increase the reasoning effort, or combine both. Which combination is best depends on the desired accuracy, cost, and latency.

So far, the article should give you a pretty solid understanding of how reasoning effort modes work and how they are implemented. This is a fine point to wrap the article if you are short on time. Otherwise, if you want to look into some of the nitty-gritty details of some of the recent open-weight models, please read on!

6. Bonus: Different ways to implement reasoning efforts (in flagship open-weight LLMs)

[This section is fine to skip unless you are interested in some additional details]

Section 5 described two possible ways to train reasoning-effort controls, namely effort-conditioned supervised fine-tuning and reinforcement learning with different token costs. Originally, I wanted to cover research articles on alternative ways to implement reasoning budgets. However, reading through most of these articles, they seemed more like proofs-of-concept that may or may not work well in practice.

So, instead of covering those, I decided to pivot a bit and cover those recipes used in state-of-the-art and notable open-weight (flagship) LLMs. For these models, there is at least evidence that the methods work in practice.

This leaves six examples. DeepSeek V4, Nemotron 3 Ultra, Kimi K2.5, GLM-5, Qwen3 and Inkling. They have different levels of detail in their reporting, but each contribute a useful variation. (I exclude models whose reports only show an effort setting in the user interface without explaining how that behavior was trained.)

6.1 DeepSeek V4 trains separate effort specialist

Let's start with the [DeepSeek V4 technical report](#), which describes the use of three modes:

- **Non-think** produces a direct response without a reasoning trace.
- **Think High** is the classic approach where the model places the reasoning trace between `<think>` and `</think>` tags. This is similar to what was discussed in the DeepSeek R1 section (section 2) at the beginning of this article.
- **Think Max** is the same as above but adds a special system instruction. (More on that below.)

The additional system prompt instruction for Think Max starts with "Reasoning Effort: Absolute maximum with no shortcuts permitted."

Thinking Mode Toggle and Effort Control

	Control Parameter (OpenAI Format)	Control Parameter (Anthropic Format)
Thinking Mode Toggle ⁽¹⁾	<code>{"thinking": {"type": "enabled/disabled"}}</code>	
Thinking Effort Control ⁽²⁾⁽³⁾	<code>{"reasoning_effort": "high/max"}</code>	<code>{"output_config": {"effort": "high/max"}}</code>

(1) The thinking toggle defaults to `enabled`

(2) In thinking mode, the default effort is `high` for regular requests; for some complex agent requests (such as Claude Code, OpenCode), effort is automatically set to `max`

(3) In thinking mode, for compatibility, `low` and `medium` are mapped to `high`, and `xhigh` is mapped to `max`

When using the OpenAI SDK, you need to pass the `thinking` parameter within `extra_body`:

```
response = client.chat.completions.create(  
    model="deepseek-v4-pro",  
    # ...  
    reasoning_effort="high",  
    extra_body={"thinking": {"type": "enabled"}}  
)
```

Figure 25: Reasoning effort control overview from the [DeepSeek V4 documentation](#)

At first, this sounds like a simple prompt engineering trick, but this prompt is actual backed by a different training setup. That is, each mode uses its own context window and length penalty (unfortunately, the exact length penalty implementation is not detailed in this report). Think Max receives a longer context window and a smaller length penalty than Think High, which gives it more room to continue reasoning.

So, the system instruction selects a behavior that was created during post-training. Adding the same instruction to an arbitrary model would not have the same effect.

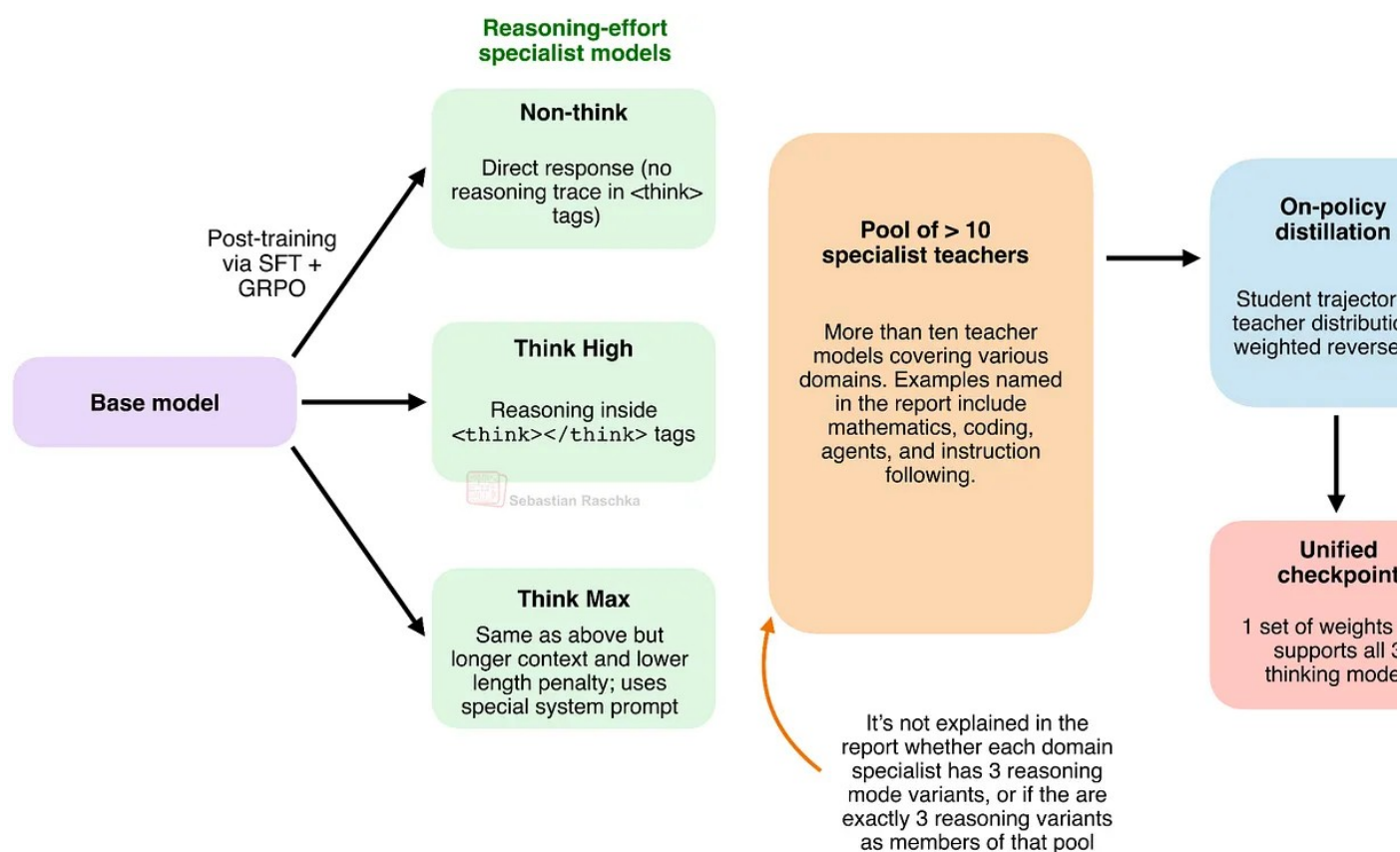


Figure 26: DeepSeek V4 describes the three effort modes and the larger teacher pool in separate parts of the report. The teacher pool contains more than ten domain specialists. The report does not disclose how these teachers map to Non-think, Think High, and Think Max.

Unfortunately, the public, and otherwise very detailed DeepSeek V4 report does not connect the descriptions of the reasoning mode and domain specialists in enough

detail to reconstruct the exact teacher assignment.

However, the report states that the final model, which supports different reasoning effort levels, was created via on-policy distillation from said teachers.

To summarize, DeepSeek V4 develops the three reasoning specialists during post-training. Starting from the base model, it applies supervised fine-tuning followed by RLVR via GRPO. The RL configuration differs for each mode. In particular, each specialist uses its own context window and length penalty, while Think Max additionally receives a special system instruction.

Then, including domain specialists, the different reasoning mode specialists are distilled into a single checkpoint that supports all three effort modes.

6.2 Nemotron 3 Ultra combines learned modes with hard budgets

The [Nemotron 3 Ultra technical report](#) describes three settings called reasoning-off regular, and medium-effort, analogous to DeepSeek V4 in the previous section. Medium-effort is the cheaper reasoning mode compared with regular. NVIDIA introduces this mode during SFT using examples generated by GPT-OSS-120B in its medium-effort mode, and then further optimizes it during RLVR. About 2.5% of the RLVR prompts use medium-effort (this corresponds to length-based adjustments applied to their rewards).

6.2.1 Using Nemotron reasoning budgets during inference

At inference time, all three modes are selected through the [chat template](#).

```
Python Copy  
  
from openai import OpenAI  
client = OpenAI(base_url="http://localhost:8000/v1", api_key="EMPTY")  
MODEL = "nvidia/nemotron-3-ultra"
```

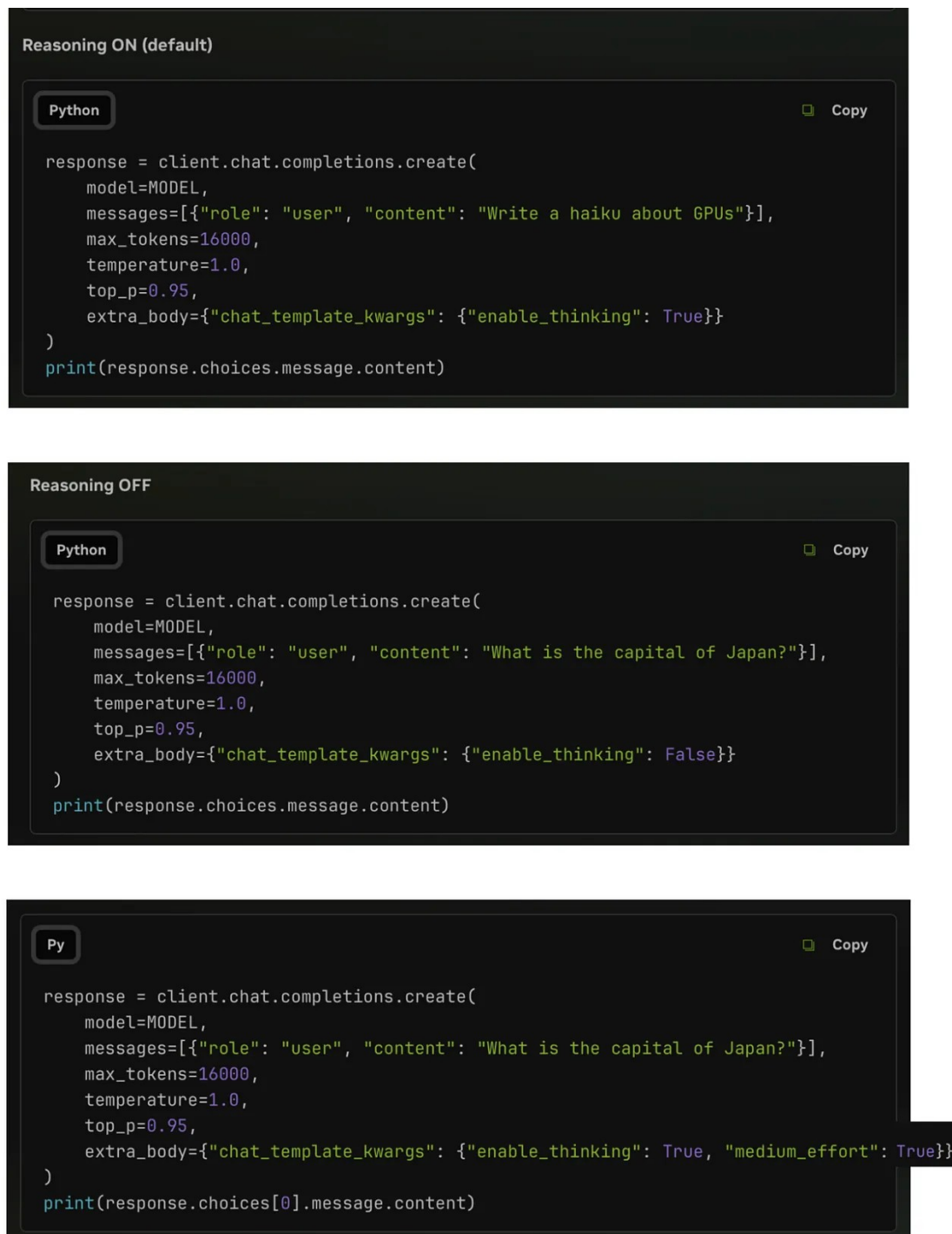


Figure 27: Nemotron 3 Ultra reasoning settings via the chat template (examples from the [official model card](#))

- 1) Regular is the default and uses `enable_thinking=True`, which starts the assistant response with an opening `<think>` tag.
- 2) Medium-effort uses `enable_thinking=True` together with `medium_effort=True`

where the latter setting also appends {reasoning effort: efficient} to the latest user message.

By the way, to further complicate things, the regular and medium-effort modes can also be combined with a separate inference-time reasoning budget. This budget acts as an external stopping mechanism. In the released implementation, the chat client asks the model to end the reasoning trace near the chosen token limit. If the model has not emitted `</think>`, the client closes the reasoning block and continues generation to produce the final answer. The learned effort mode determines how the model uses its reasoning tokens, while the budget constrains how long the reasoning trace can continue. This makes it possible to pair either mode with a tighter or looser budget depending on the desired cost and accuracy.

3) Reasoning-off uses `enable_thinking=False`, which prefills an empty `<think><think>` block (similar to Qwen3 discussed in section 4) so that the model proceeds directly to the final response. Thus, these are chat-template controls rather than system prompts.

6.2.2 Reasoning budget-aware training in Nemotron

The inference controls described above are backed by two related SFT components. The first introduces medium-effort behavior using GPT-OSS-120B traces, as discussed earlier. The second prepares the model for hard reasoning budgets.

To construct this training data, the authors take regular reasoning traces, truncate them at randomly selected token budgets, and keep the original final answers. The inserted `</think>` token is masked from the SFT loss. As a result, the model sees examples where it has to move from an incomplete reasoning trace to the answer as if the reasoning block has been closed externally.

Medium-effort training then continues during RLVR. About 2.5% of the RL prompts use the medium-effort setting across math, STEM, and coding tasks. The report notes that

the mode can be calibrated through reward hyperparameters where length-based reward adjustments provide additional control over the cost-quality trade-off.

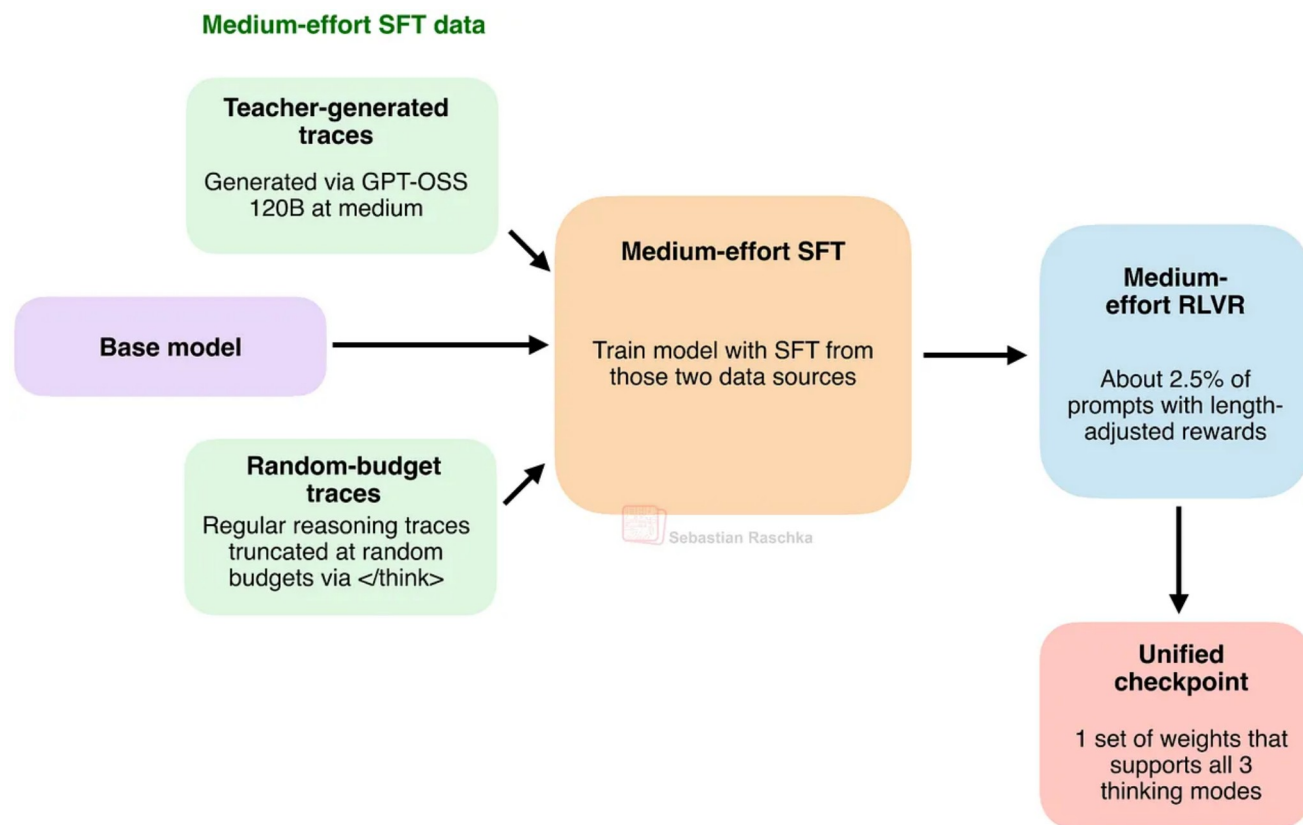


Figure 28: Nemotron 3 Ultra introduces medium effort with teacher-generated SFT data, random-budget truncation, and a small medium-effort subset during RLVR.

6.3 Kimi K2.5 alternates budgeted and unconstrained RL

The [Kimi K2.5 technical report](#) discusses a training method called Token Efficient RL lower reasoning effort. (While there was a K3 announcement this week, the reasoning effort methodology of K3 is not publicly disclosed, but it could be similar or related K2.5.)

6.3.1 Kimi's Toggle method

The report mentions that a fixed token budget can make a reasoning model overfit short solutions. That means the model becomes more concise (i.e., faster and cheap

but it may lose the ability to benefit from additional inference-time compute and cā thus perform poorly.

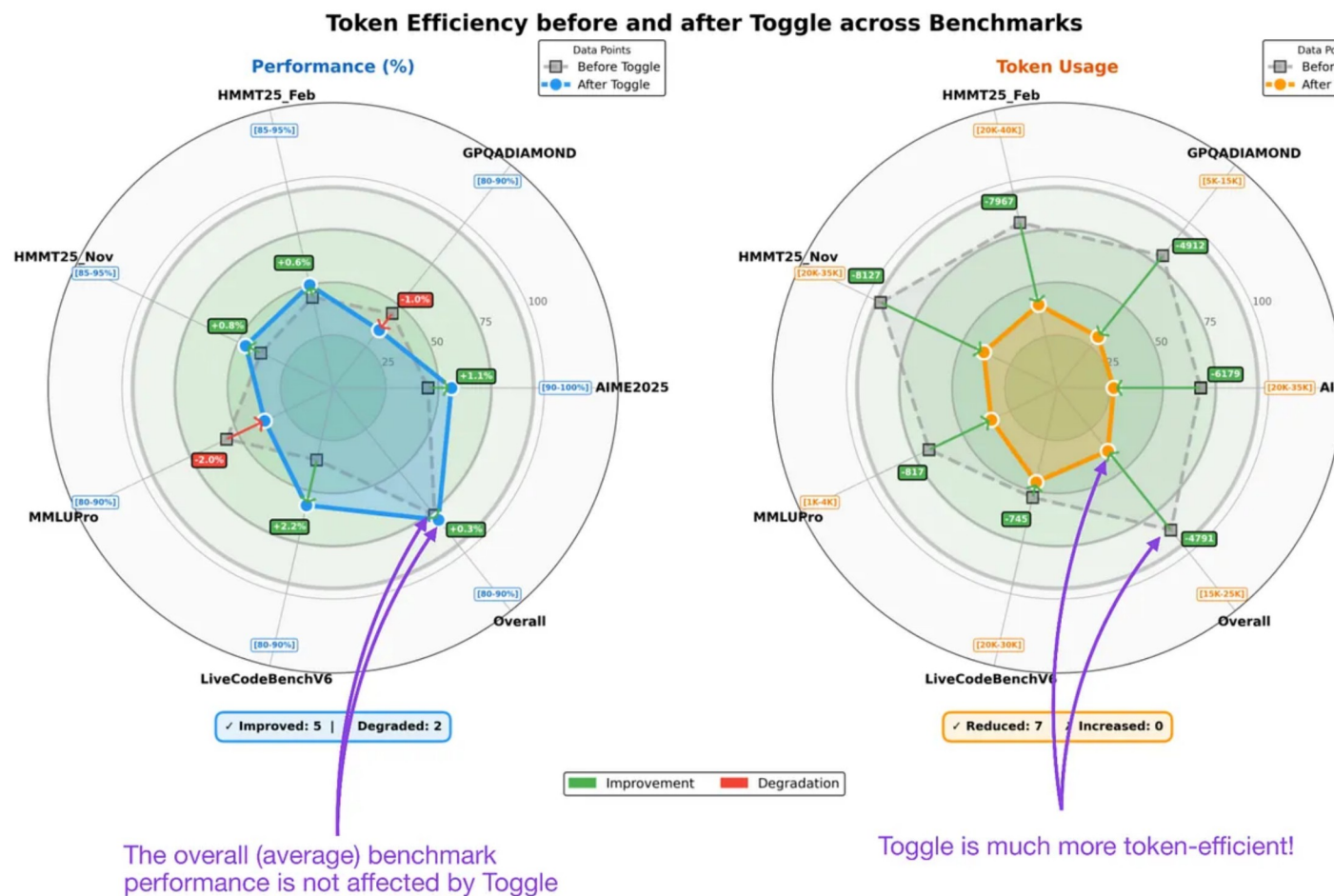


Figure 29: The proposed Toggle method makes Kimi K2.5 much more token-efficient while keeping the overall benchmark performance similar. Annotated figure from <https://arxiv.org/abs/2602.02276>

Kimi K2.5's method, called Toggle, alternates between two RL phases every fixed number of training iterations:

1. In the budgeted phase, correct solutions are encouraged to stay within a problem specific token budget.
2. In the unconstrained phase, the usual maximum generation length is restored so that the model can still learn from longer solutions.

For each problem, the budget is estimated from a selected percentile of response

lengths among correct rollouts in RLVR. The budget constraint is then only activated once the mean accuracy on that problem exceeds a threshold. This avoids forcing the model to shorten its reasoning before it can solve the problem reliably.

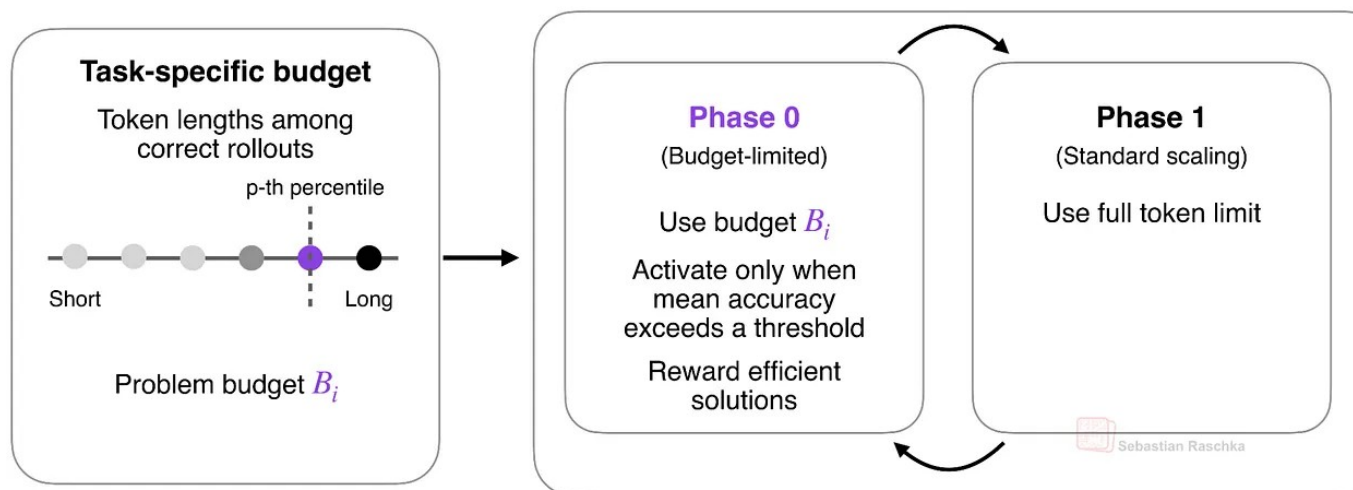


Figure 30: Overview of the two phases of the Toggle method.

The report evaluates Toggle on K2 Thinking and finds that it reduces generated tokens by about 25 to 30% with little change in benchmark performance. The same behavior also transfers from math and coding RL tasks to GPQA and MMLU-Pro.

Toggle supplies a concrete flagship-model recipe for training a more token-efficient reasoning policy while preserving its ability to scale at test time.

6.3.2 What Toggle changes at inference

Toggle operates entirely during RL training. Both alternating phases update the same policy (i.e., LLM), and the final (unified) checkpoint has no budgeted-versus-unconstrained selector. At inference, the resulting model then runs in thinking mode by default.

Interestingly, though, Kimi K2.5 itself exposes a separate binary choice between thinking and instant modes in some APIs I checked (like vLLM or SGLang). Thinking mode is enabled by default. Instant mode disables the reasoning trace through

thinking: {"type": "disabled"} in the official API or `chat_template_kwargs={"thinking": False}` when serving the model through vLLM or SGLang. However, these settings are separate from Toggle.

Also, the official Kimi report does not provide a separate training recipe for instant mode. However, K2.5's SFT data were generated using both the earlier K2 model, which produces direct responses without long reasoning, and K2 Thinking, which produces extended reasoning traces. This likely exposes the unified checkpoint to both response formats similar to what's done in Nemotron 3 above. At inference time, the chat template selects between them by prefilling either an open `<think>` tag for thinking mode or an empty `<think></think>` block for instant mode. But again, unfortunately, the report does not disclose the exact data mixture or whether additional mode-specific RL was used.

The newer Kimi K3 provides a more direct inference-time effort interface. The [current Kimi Code documentation](#) lists three settings called low, high, and max, with max as the default. These are passed through the `reasoning_effort` parameter. However, Moonshot has not yet explained how the three effort levels were created during training. Its [launch post](#) says that these details will appear in a future K3 technical report, so I'll stay tuned for that.

6.3.3 Kimi K3 (Update)

Based on the new [Kimi K3 technical report](#) (released after I published this article), Kimi K3 has three reasoning modes: low-, high-, and max-effort.

For each training problem, the researchers first estimate an initial token budget. On a simple verifiable task, a correct response would normally receive a reward of +1 and an incorrect response 0. But here, if the response exceeds the selected token budget, the reward is instead set to -1. This gives the model a strong incentive to stay within the budget. (The exact reward setup is different for non-verifiable and agentic tasks, and Kimi also uses learned reward models and pairwise comparisons.)

To have Kimi K3 support the three reasoning effort modes, the training looked as follows. First, the researchers start by training a max-effort version ("specialist") with relatively generous budget. They then lower the budget to train high- and low-effort versions.

This process is repeated for three domains: general tasks, general agents, and coding agents. And each domain gets one specialist for each effort level. So, for example, one specialist learns low-effort coding behavior, while another learns max-effort agent behavior. This gives nine specialist models in total.

The behaviors of these nine specialist models are then combined into a single Kimi model using multi-teacher on-policy distillation. At inference time, a natural-language thinking-effort instruction in the prompt can then be used to tell the model which effort level to use.

6.4 GLM-5 introduces turn-level and interleaved thinking through SFT

The [GLM-5 technical report](#) extends the binary on/off thinking switch introduced with GLM-4.5 to multi-turn and tool-using scenarios. It describes three related behaviors (rather than three effort levels):

- **Interleaved thinking:** this inserts a reasoning block before each response and tool call.
- **Preserved thinking:** here, the chat retains earlier reasoning blocks across turns so that the model can reuse them later.
- **Turn-level thinking:** this enables or disables reasoning separately for each request in a conversation.

At inference time, turn-level thinking is the actual on-off switch. In the [Z.ai API](#), thinking is enabled by default and can be disabled for an individual request with

thinking: {"type": "disabled"}. The hosted implementation is not disclosed but open [GLM-5 chat template](#) shows the equivalent mechanism when self-hosting with Transformers, vLLM, or SGLang.

It starts the assistant response with `<|assistant|><think>` when thinking is enabled and `<|assistant|></think>` when it is disabled. The latter closes the reasoning block immediately, so generation proceeds directly to the final answer.

The report says that these behaviors are introduced during multi-task SFT together with an updated chat template.

After SFT, GLM-5 goes through reasoning RL, agentic RL, and general RL. And a final on-policy distillation step uses checkpoints from the preceding stages as teachers. This helps the final model recover capabilities that may have weakened during the sequential RL stages.

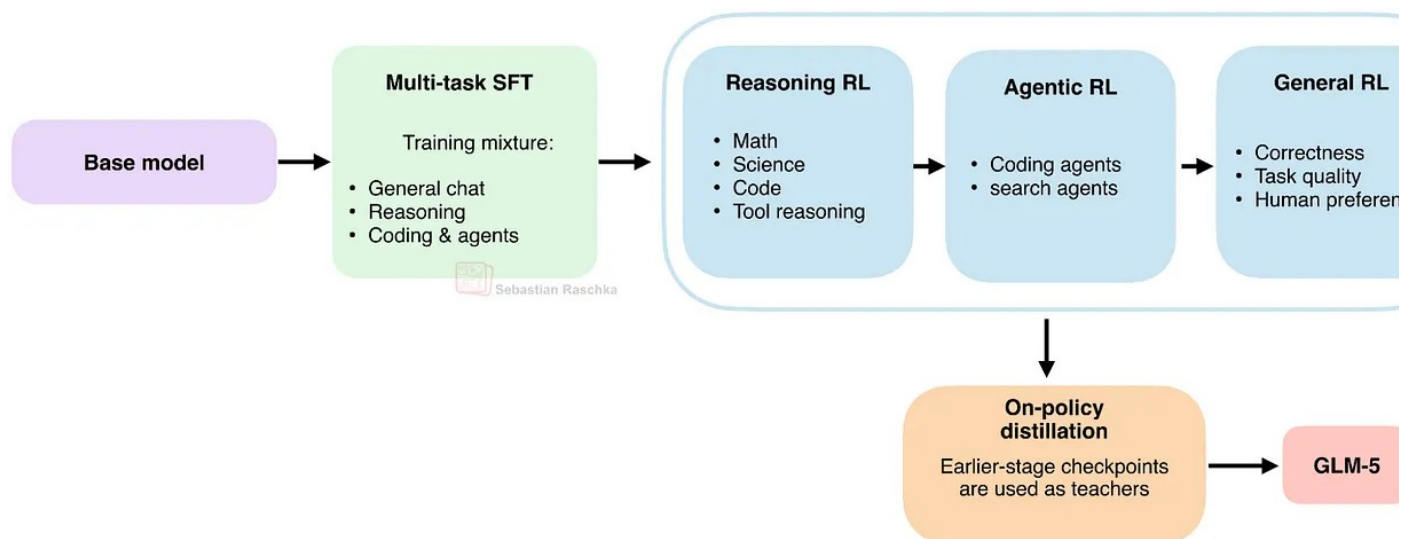


Figure 31: GLM-5 training pipeline.

6.5 Qwen3 uses mode fusion and inference-time truncation

Qwen3 was already covered in Section 4, so I will only summarize the parts that matter for this comparison. According to the [Qwen3 technical report](#), its post-training pipeline has four stages. These are long-chain-of-thought SFT, reasoning RL, Thinking Mode Fusion, and general RL.

Thinking Mode Fusion is the key stage for the effort on-off switch. Here the model is trained via SFT on a mixture of thinking and non-thinking examples. The `/think` examples contain a reasoning trace, while `/no_think` examples begin with an empty `<think></think>` block that is accompanied by a short answer. The following general RL stage reinforces instruction and format following for both behaviors.

Qwen3 also supports a hard thinking budget. At the requested threshold, the reasoning span is stopped and a stop-thinking instruction is inserted before the model continues with its final answer. The report says that this partial-reasoning behavior was not trained explicitly. It emerged after Thinking Mode Fusion.

This gives Qwen3 a learned on-off switch plus an inference-time budget. It is similar but simpler than the DeepSeek V4 and Nemotron recipes.

6.6 Inkling conditions RL on a continuous effort value

Inkling was already discussed in Section 5.3. The short version is that its [technical report](#) mentions that they use continuous effort conditioning (values between 0.0 and 1.0) rather than fixed effort labels.

After a relatively small initial SFT stage, most of Inkling's post-training comes from asynchronous RL with more than 30 million rollouts. The desired effort is included in the system message, and the token length penalty is adjusted according to that value during RL. As previously discussed, a higher token cost encourages a shorter response. A lower token cost gives the model more room to reason.

6.7 Overview of the known recipes

The table below summarizes what is actually documented in the six technical report

Model	Effort control	Disclosed training mechanism	Inference control
DeepSeek V4	Non-think, High, Max	Separate specialists with mode-specific SFT, GRPO, context windows, and length penalties, followed by on-policy distillation	Response format and a Max system instruction
Nemotron 3 Ultra	Off, medium, regular	Teacher-generated medium traces, random-budget truncation, and length-adjusted RLVR	Learned mode plus an optional hard budget
Kimi K2.5	Implicit token-efficient reasoning	The Toggle mechanism alternates budgeted and unconstrained RL using problem-specific budgets	Thinking and instant modes exist, but Toggle is not exposed as a separate selector
GLM-5	Thinking on or off per turn, plus interleaved and preserved thinking	Multi-task SFT with an updated chat template, sequential reasoning, agentic, and general RL, followed by on-policy cross-stage distillation	Binary per-turn switch, automatic reasoning before tool calls, and optional retention of earlier reasoning blocks
Qwen3	No-think, think, token budget	Thinking Mode Fusion SFT followed by general RL	/no_think, /think, and inference-time truncation
Inkling	Continuous scalar (0.0 to 1.0)	Effort-conditioned RL with an effort-dependent token cost	Effort value in the system message

Figure 32: Comparison of the disclosed training mechanisms and inference controls for six open-weight models with reasoning-effort settings.

So, looking at the six different open-weight models, they have a shared framework. First, they introduce effort mode control through SFT and the chat template. Qwen3 explicitly mixes thinking and non-thinking examples, while GLM-5 adds interleaved, preserved, and turn-level thinking patterns.

The second shared component is a mode-conditioned RL stage, where context windows and length penalties change with the requested effort. DeepSeek V4, Nemotron 3 Ultra, and Inkling use this approach.

A third ingredient improves robustness under explicit budgets. Nemotron trains on randomly truncated traces, Qwen3 can continue from a forcibly stopped reasoning span, and Kimi alternates budgeted with unconstrained RL. These methods help preserve answer quality when the available reasoning length changes and is even cut short.

7. Conclusion

The open-weight examples in this article implement reasoning effort through several different mechanisms. Similar labels can be backed by separate specialists, mixed SFT data, mode-conditioned rewards, hard token budgets, or combinations of these methods.

It is difficult to say which approach is best. The models differ in their base checkpoints, training data, post-training compute, benchmarks, and serving goals. Their reports often omit many details needed for a controlled comparison. (Also, there may not be a one-size-fits-all, and a method that works well for an interactive assistant may be a poor fit for a long-running coding agent.)

The holy grail is of course automatic effort selection. We saw this a while back with GPT 5's Auto mode. It's a tricky problem to solve, and in the end, the implementation was probably more miss than hit, which is why it got removed from the UI (at least,

can't find it anymore).

In the near future, I think reasoning effort will remain an explicit model input, which will most often be delivered through the system prompt. However an agent wrapper/harness around the LLM, or an internal router may increasingly infer appropriate mode and budget from the task state and available resources automatically (while of course still allowing a user override).

I still hope that effort selection will become more automatic. Similar to GPT 5's auto mode, a cheap model or router could choose the mode from the request, tool state and remaining time or token budget while still allowing a user override. The override is useful if you want to optimize for latency or cost, or maximum performance.

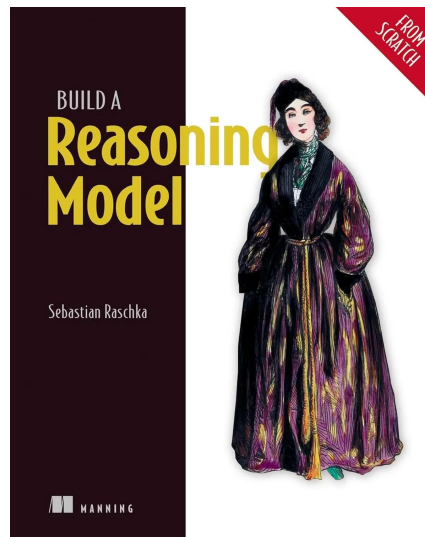
I realize that this was a long article, and it was perhaps not the flashiest topic. But I thought that given all the talk about LLMs, reasoning models, and agents, a look at reasoning models was something not covered before, and I hope it was a unique and somewhat useful overview!

Further resources

If you want a hands-on implementation of the core training methods behind reasoning models, my [Build a Reasoning Model \(From Scratch\)](#) book walks through reinforcement learning with verifiable rewards and inference-time scaling step by step with code.

This article focused on how a trained reasoning model can support different effort modes. The book takes a step back and shows how to turn a conventional LLM into a reasoning model in the first place. It is a sequel to [Build a Large Language Model \(From Scratch\)](#) and starts where that book leaves off.

The print edition has now started shipping



Build a Reasoning Model (From Scratch) [Manning] [Amazon]

If you liked my previous [Build a Large Language Model \(From Scratch\)](#) book, this is essentially a sequel implementing inference-time scaling techniques and reinforcement learning algorithms from scratch.

And thanks so much for being a paid subscriber, your support helps tremendously to keep writing these independent deep dives and sharing the accompanying code, figures, and experiments.



Recommend Ahead of AI to your readers

Ahead of AI focuses on machine learning and AI research and is read by more than 200,000 researchers and practitioners who want to stay ahead in a rapidly evolving field.

Recommend



349 Likes · 32 Restacks

Discussion about this post

Comments

Restacks



Write a comment...



Denis Lemarchand Jul 18

Liked by Sebastian Raschka, PhD

Hi Sebastian,

I just wanted to drop you a quick note to say how much I loved this article, as always! It provides fantastic synthesis, especially as a window into your book *Build a Reasoning Model (From Scratch)*. Since I frequently read your work, I wanted to share my typical routine for absorbing your article.

As a French speaker, I often use my browser's built-in translation tool for the first read. It allows me to quickly grasp the core essence of the article without getting slowed down by language nuances.

Once I'm done reading, I turn to my favorite AI agent (or whichever one I am currently exploring) and ask it to generate a 10-question multiple-choice quiz based on your text, followed by 2 open-ended essay questions. My goal isn't just to validate that I understood the material, but to ensure I can confidently explain these concepts to my colleagues. I work in applied AI, building agentic use cases, and this has proven to be an excellent strategy for continuous learning. It keeps me sharp and ensures I don't become passive or "enslaved" by the technology.

This actually made me wonder: What is your personal strategy for continuous learning? More specifically, how do you keep yourself from succumbing to convenience and falling victim to automation bias (like relying too heavily on AI summaries to write or research your own articles)?

Lastly, I've been seriously considering your paid subscription. My only hesitation is that my time is limited, and I realistically won't be able to keep up with your incredibly prolific output of books and articles! Beyond the excellent deed of supporting your work, what would you say is the key value gain of the subscription for someone in my position?

Thank you again for your amazing contributions to the community.

Best regards,

Denis

♡ LIKE (5) 💬 REPLY



1 reply by Sebastian Raschka, PhD

© 2026 Raschka AI Research (RAIR) Lab LLC · [Privacy](#) · [Terms](#) · [Collection notice](#)
[Substack](#) is the home for great culture

Fantastic article, thank you for sharing your seemingly deep knowledge about this domain in a way even "laymen" like myself can learn and appreciate.

♡ LIKE (3) 💬 REPLY



1 reply by Sebastian Raschka, PhD

36 more comments...